

2026

Inertia AI

בית ספר: הר וגיא

ת.ז: 217535541

מגיש: מעיין אושרי

תוכן עניינים

2	מבוא ומוטיבציה:
2	מטרות המערכת ויעדים טכנולוגיים:
3	מדריך למשתמש ותזרים תפעולי:
6	גרסאות חומרה תוכנה והרשאות
6	תכנון ומימוש המערכת: עיבוד אותות דיגיטלי וסינון רעשים
9	ארכיטקטורת התוכנה ועיבוד נתונים ברקע
9	1. שכבת מנוע החיפוש (Engine) ועיבוד הרקע
15	3. שכבת התצוגה (UI Layer) ותקשורת בין תהליכים (IPC)
15	4. שכבת מסד הנתונים והענן: ניהול מידע אסינכרוני ב-Google Cloud Firestore
20	שכבת ההתחברות והרשמה (auth)
26	אתגרי פיתוח ופתרונם
27	סיכום רפלקטיבי וכיווני פיתוח עתידיים
27	קטעי קוד

מבוא ומוטיבציה:

בעשור האחרון חלה מהפכה משמעותית בעולם הניטור הגופני והמדדים הפיזיולוגיים בספורט. רצים, אתלטים ומתאמנים לא מקצועיים נעזרים במגוון רחב של מוצרים לבישים כגון: שעונים ורצועות דופק. מרבית הכלים המצויים בשוק כיום מתנהגים כ"קופסא שחורה", הם מספקים למשתמש את השורות התחתונות של ביצועיו ומציגות לו נתונים סופיים בלבד - לדוגמא שריפת הקלוריות המוערכת היומית, מרחק ומספר צעדים שנצבר במהלך היום – זאת מבלי לאפשר גישה לנתונים הפיזיקליים הגולמיים שנאספו מהמערכת וללא הצגת הפיזיקה והביומכניקה העומדות מאחורי התנועה בזמן אמת.

מניסיוני החוסר הזה מורגש ביתר שאת אצל מתאמנים המכילים את עצמם למבחנים פיזיים עצימים ומיונים מורכבים כדוגמת ימי סיירות וגיבושי יחידות עילית (גיבוש מטכ"ל-יאללה, שייטת-חלשים וכדומה). במסגרות הכנה אלו, עצימות האימונים גבוהה במיוחד, והשמירה על טכניקה נכונה, קצב פסיעות מבוקר (Cadence) ומנח גוף אופטימלי תחת עייפות קיצונית הכרחיות מאוד על מנת למנוע פציעות וכמו כן לייעל את המכניות של הגוף.

המערכת Inertia-AI נוצרה מתוך חיבור מעשי בין עולם האימון הפיזי לבין תחומי המדעי המחשב ופיזיקה היישומית. הפרויקט בא במטרה לפתוח את "הקופסא השחורה" ולהפוך את המכשיר טלפון של המתאמן למעבדה ביומכנית ניידת ונגישה שמנצלת את כלל החיישנים הנגישים בחלק נרחב מאוד מהטלפון החכמים הנגישים כיום בשוק, המערכת חוקרת את הביופיזיקה של התנועה לעומק, תוך הפעלת עיבוד מקומי על הנתונים ממהלך התנועה וכמו כן מספקת משוב.

מטרות המערכת ויעדים טכנולוגיים:

פיתוחה של המערכת הונחה על ידי ארבע מטרות ליבה מרכזיות, המשלבות בתוכן צרכים פיזיולוגיים וכמו כן פתרונות פיתוח תוכנה ופיזיקה מתקדמות.

- **ניתוח ביומכני רציף ומדויק:** חילוץ זווית היציבה והטיית הגוף בזמן אמת באמצעות היתוך חיישנים ומסנן משלים, על מנת לזהות שגיאות ביציבה, וכמו כן שימוש במד התאוצה הלינארית על מנת לזהות מידע על התקדמות בציר האופקי.
- **עיבוד אותות על מכשיר עם מערכת מוטבעת (Embedded system):** עיבוד אותות זו משימה כבדה מבחינת כוח חישוב ולכן נדרשת דרך לחלק את כוח העיבוד הזה בין כמה ליבות, החישובים כוללים התמרת פורייה הבדידה (DFT) לשם בידוד התנועה של הצעד משאר הרעשים הנוצרים על ידי תזוזות שאינן קשורות.
- **ניהול נתונים מאובטח ומבוצר בענן:**

יצירת צנרת מאובטחת המגבה כל אימון שעובד בצורה אסינכרונית בבסיס הנתונים Google Cloud Firestore, תוך כדי ניהול זהויות מאובטח מול Firebase Authentication.

- **הנחת תשתית לתיוג של נתוני אימון**
הפלטפורמה של האפליקציה מאפשרת לייצר בסיס נתונים מתויג, אשר בעתיד יכול לשמש לאימון מודלי למידת מכונה (Random forest).

מדריך למשתמש ותזרים תפעולי:

המערכת בנויה בצורה לינארית ואינטואיטיבית המותאמת לתנאי אימון. שלבי התפעול הם:

1. **משתמשים חדשים:** יש לגשת למסך הרישום (SignUpActivity) ולהזין כתובת דוא"ל וסיסמה העומדים בתקני האבטחה הסטנדרטיים. המערכת תפתח פרופיל משתמש ייעודי בענן.
2. **משתמשים קיימים:** יש לגשת למסך ההתחברות (LoginActivity) ולהזין את פרטי הגישה. התהליך מתבצע בצורה מוצפנת ומאובטחת ישירות מול שרתי Google Firebase.

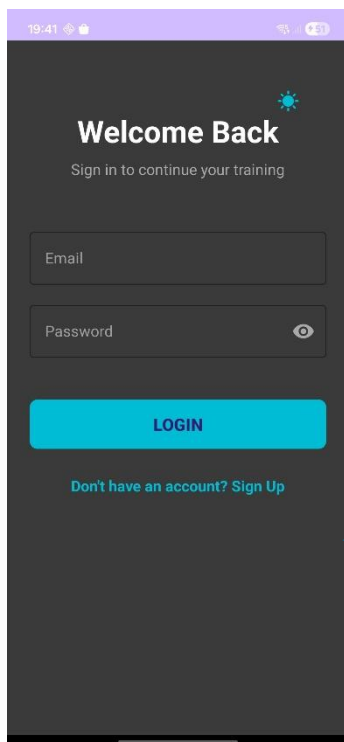
לאחר שלב האימות המוצלח יופיע העמוד הראשי של האפליקציה המציג עיצוב נקי ולמסך יש שני כפתורי פעולה:

- **Live Sensor Tracking** לחיצה על כפתור זה תעביר את המשתמש לביצוע של אימון, בתוך מסך זה יוצגו נתונים שמחושבים בזמן אמת אופציה לביצוע איפוס של מכשיר המדידה והנתונים שנמדדו כבר וכמו כן גם כפתור לשמירת האימון והעלאתו לבסיס הנתונים שנמצא בענן.
- **Workout history** מעביר את המשתמש להצגת נתוני האימון הקודמים שביצע, בנתונים מופיע מידע אודות הצעדים שנעשו, קצבם תאריך האימון ועוד.

תרשימי זרימה של האפליקציה:

משתמש שנרשם:

משתמש קיים:



19:41

Welcome Back 

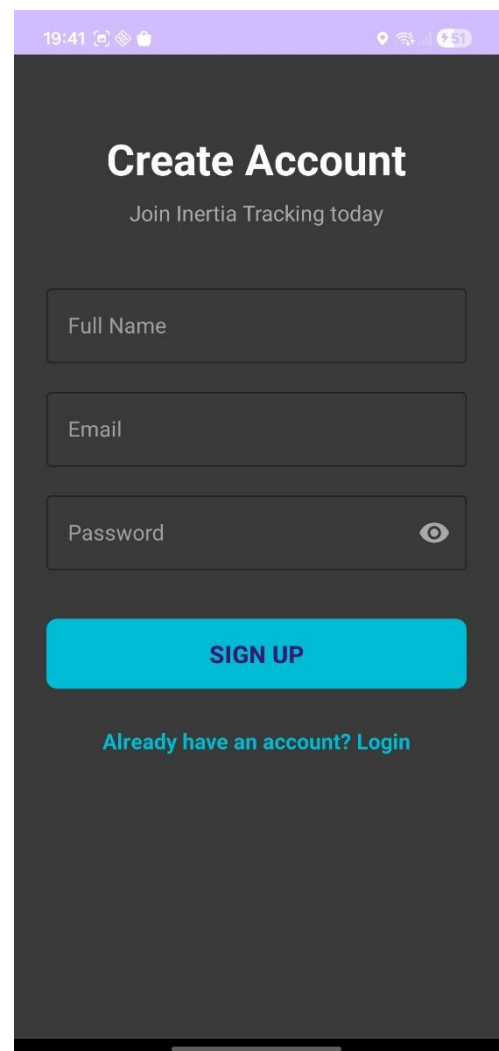
Sign in to continue your training

Email

Password 

LOGIN

Don't have an account? Sign Up



19:41

Create Account

Join Inertia Tracking today

Full Name

Email

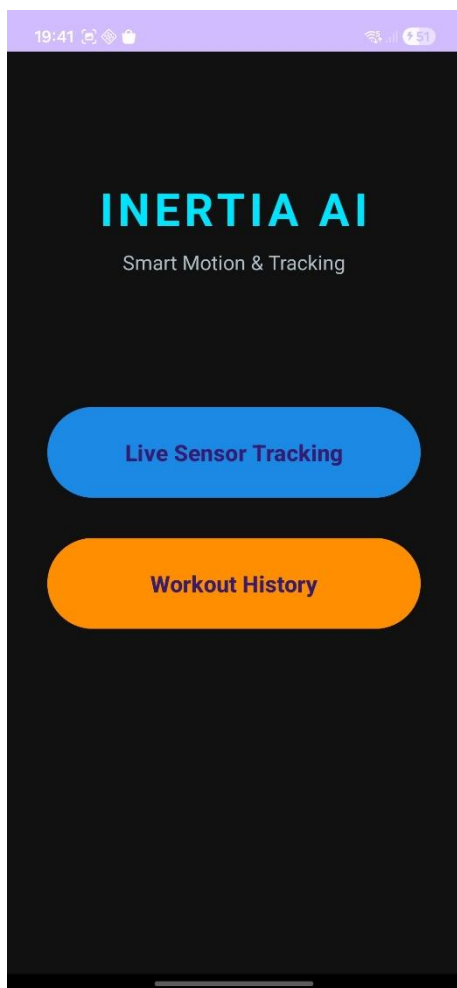
Password 

SIGN UP

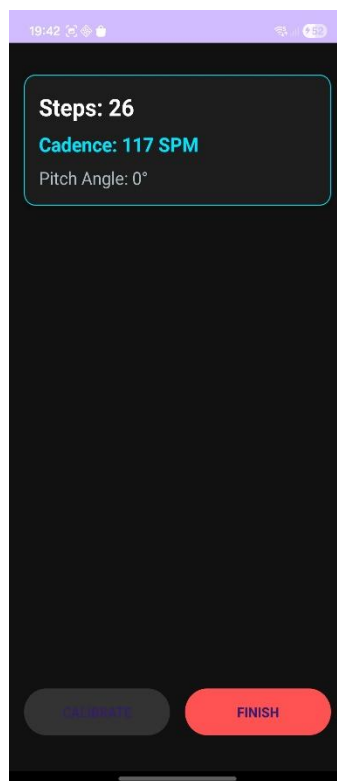
Already have an account? Login

נרשם בהצלחה

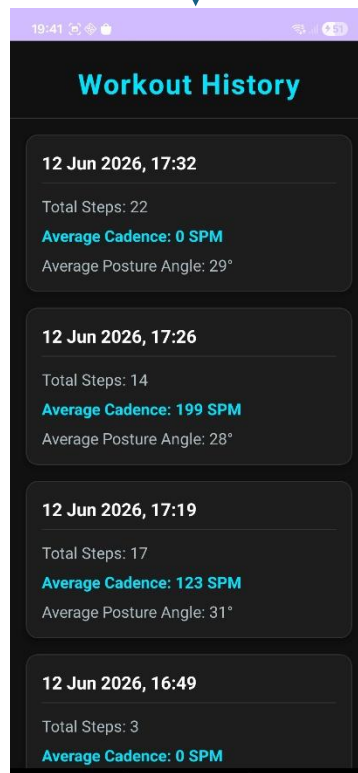
התחבר בהצלחה



ביצוע אימון



התבוננות באימונים



גרסאות חומרה ותוכנה והרשאות

דרישות חומרה: מכשיר אנדרואיד בעל חיישנים אינרציאליים מובנים (מד תאוצה וגירוסקופ).

גרסאות תוכנה: פיתוח ב-Android Studio (Java 8), תאימות לגרסאות אנדרואיד Min SDK 24 ועד Target SDK 34.

שירותי ענן: חיבור חובה לאינטרנט לצורך אימות משתמשים ושמירת נתונים (Google firebase).

```
<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_HEALTH" />
<uses-permission android:name="android.permission.HIGH_SAMPLING_RATE_SENSORS" />
<uses-permission android:name="android.permission.INTERNET" />
```

כפי שניתן לראות התוכנה דורשת מהמשתמש מספר הרשאות:

שירותי רקע:

ההרשאה הראשונה שמופיעה היא אפשרות של שירותי רקע כפי שיוסבר בהמשך, האפליקציה ניתנת להפעלה גם כאשר הטלפון לא מופעל, אלא מכובה. ההרשאה השנייה היא מסירת המידע למערכת אנדרואיד- פיצ'ר חדש, לשם מה התהליך רקע משמש במקרה פה הוא עוסק בספורט וניתור הבריאות.

קצב דגימה גבוה:

ההרשאה השלישית קשורה בקצב דגימות גבוה מהרגיל מחיישני המכשיר, על מנת ליישם את אלגוריתם עיבוד האותות שיוסבר בהמשך לפעול בצורה מיטבית.

גישה לרשת האינטרנט:

"הרשאה סטנדרטית אך הכרחית", המאפשרת לאפליקציה לפתוח סוקטים ולגשת לאינטרנט, בלעדיה האפליקציה מנותקת ואינה יכולה לתקשר עם שירותי הענן לצורך אימות משתמשים, העלאת נתוני אימון או שליפתם.

תכנון ומימוש המערכת: עיבוד אותות דיגיטלי וסינון רעשים

אחד האתגרים המרכזיים בפיתוח מערכת ניטור תנועה מבוססת סמארטפון הוא התמודדות עם "רעש" סביבתי. כאשר המכשיר נמצא בכיסו של המתאמן, מד התאוצה לא קולט רק את הצעדים עצמם, אלא גם זעזועים ממשטחי הריצה, רעידות לא קשורות הנובעות מפעילות גופנית ועוד.

Thresholding (פסילת גישת הסף):

הגישה הנאיבית לזיהוי צעדים מבוססת על חיפוש נקודות קיצון מקומיות בגרף התאוצה שחוצות סף מוגדר מראש. עם זאת, גישה זו הוכחה כרגישה מדי לרעשים ואינה מתאימה לאימונים עצימים שבהם עוצמת הזעזוע משתנה.

(DFT) הפתרון האלגוריתמי: התמרת פורייה בדידה

כדי לבדוד את הצעד האמיתי מתוך כלל הרעשים בסביבה המערכת ממירה את הנתונים הגולמיים ממישור הזמן למישור התדר באמצעות התמרת פורייה הבדידה אלגוריתם זה מאפשר לנו לפרק את האות המורכב של מד התאוצה למרכיביו ההרמוניים (אלו גלי סינוס וקוסינוס) וכך מאפשר למצוא את התדר הדומיננטי ביותר שהוא בעצם קצב הפסיעות.

לעומק הדברים:

נגדיר פונקציה הרמונית (בהגדרה היבשה היא פונקציה שמקיימת את משוואת לפלס-משוואה דיפרנציאלית חלקית שפונקציות "מחזוריות" מקיימות) שצורתה

$$f_{\omega}(t) = Ae^{\pm i\omega t} = A(\cos \omega t \pm i \sin \omega t) \quad \text{הכללית שלה היא:}$$

זו פונקציה בעלת תדר ω ולכן צירוף לינארי של מספר תדרים כאלה ייתן:

$$f_s(t) = \frac{1}{T} \sum_k f_{\omega_k}(t) \quad \text{מכיוון המרחב התדרים היא מערכת אורתונורמלית- שזה אומר שעבור המכפלה הפנימית המוגרת כך בין שתי פונקציות מרוכבות}$$

$\langle f, g \rangle = \int_{\alpha}^{\alpha+T} f(x) \overline{g(x)} dx$ תיתן עבור כל שתי פונקציות שונות מהמרחב הזה תוצאה אפסית ומכפלה פנימית כזו של פונקציות זהות תיתן 1 והמערכת הזו היא סגורה כלומר ניתן לייצג כל פונקציה על ידי צירוף לינארי של פונקציות ממרחב התדרים כלומר: $f(x) = \sum_{m=-\infty}^{\infty} c_m e^{imx}$ ולכן כאשר נפעיל מכפלה פנימית בין $f(x)$ לבין e^{inx} לאחר פישוטים נקבל את המקדם c_m !

כלומר סה"כ קיבלנו שהמקדם של התדר ω שמרכיב את הפונקציה שלנו הוא:

$$c_{\omega} = \langle f(x), e^{inx} \rangle = \frac{2}{b-a} \int_a^b f(x) e^{-\frac{2\pi i x}{b-a}} dx$$

(המקדמים עבור כל המכפלות האלו נובעות מסיבות שלא אפרט כאן) מה שקיבלנו הוא יכולות להבין את התדר הדומיננטי מאחר והתמרת פורייה הבדידה במקרה שלנו עבור ערכים בדידים היא:

תדירות k (נתון על ידי ההתמרה $X[k]$) נתון על ידי ביצוע הסכימה הזו, אנחנו $X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-\frac{i}{N} 2\pi kn}$ ובמילים פשוטות המקדם של התדר בעל

מחפשים את התדר שייתן את המקדם המקסימלי, N הוא מספר הדגימות שבוצעו וגם כאן לא אציין מדוע המקדמים הם כפי שהם (מאיפה פאי נכנס)

```
/**
 * DSP Implementation: Extracts the dominant cadence frequency from time-domain acceleration arrays.
 * Transforms raw magnitude blocks into frequency-domain spectral coefficients using an
 * optimized  $O(N \cdot K)$  Discrete Fourier Transform (DFT) tuned explicitly for human locomotion frequencies (0.5Hz - 4.0Hz).
 *
 * @param arr The array of raw magnitude scalars.
 * @param numberOfMagnitudeSamples The exact window frame size (N).
 * @param numberOfSamplesPerSeconds The hardware sampling rate (Hz).
 * @return Calculated cadence value in Steps Per Minute (SPM).
 */
```

```
private int getCadence(Double[] arr, int numberOfMagnitudeSamples, int numberOfSamplesPerSeconds) {
    double maxCoefficient = 0.0, currentMagnitude;
    double maxK = 0.0;

    // We use a predefined factor to eliminate redundant division operations inside the heavy  $O(N \cdot K)$  loop
    double preCalculatedFactor = (2.0 * Math.PI) / numberOfMagnitudeSamples;

    // Try to get the most accurate k value (amount of steps for about 5 seconds)
    // algorithm explained in depth on the project book
    for (double k = 5.0; k <= 25.0; k += 0.5) {
        double an = 0.0;
        double bn = 0.0;
        double kFactor = k * preCalculatedFactor;

        for (int n = 0; n < numberOfMagnitudeSamples; n++) {
            double angle = kFactor * n;
            an += arr[n] * Math.cos(angle);
            bn += arr[n] * Math.sin(angle);
        }
    }
}
```

```
currentMagnitude = Math.sqrt((an * an) + (bn * bn));
if (currentMagnitude > maxCoefficient) {
    maxCoefficient = currentMagnitude;
    maxK = k;
}
return (int) Math.round((60.0 * maxK * numberOfSamplesPerSeconds) / numberOfMagnitudeSamples);
}
```

וכך זה נראה בקוד אנחנו בודקים את כל התדרים מ 5 עד 25 בקפיצות של חצי, כלומר אם מספר הדגימות הוא 256 ומספר הדגימות בשנייה הוא 50 זה אומר שעברו $5 \approx \frac{256}{50}$ שניות ולכן אם בוצעו 5 צעדים במשך 5 שניות זה אומר שבוצע צעד בשנייה או 60 בדקה שזהו *Cadence*. (דוגמא להבהרה)

בקוד הדברים חושבו בעזרת פונקציות טריגונומטריות אך כמובן שזה שקול על פי זהות אוילר $Re^{i\theta} = R\cos\theta + iR\sin\theta$ וכמו כן לוקחים את הגודל של התמרת פורייה מכיוון שזה יוצא ערך מרוכב, ובודקים האם הוא נותן את המקדם הדומיננטי ביותר עד כה, וזה התדר שנבחר, ומחזירים את ה *Cadence* בסופו של האלגוריתם.

ארכיטקטורת התוכנה ועיבוד נתונים ברקע

מערכת התוכנה תוכננה תחת עקרונות מחמירים של הפרדת רשויות - SoC (*Separation of Concerns*) במטרה לבודד את הלוגיקה הפיזיקלית מהממשק הגרפי. גישה זו הכרחית באפליקציות הדוגמות חומרה בקצב גבוה, שכן ביצוע חישובים מתמטיים על ה *UI Thread*-המרכזי היה גורם ל"תקיעות" ולקריסת האפליקציה בצורה תדירה.

הפרדת חששות (Separation of Concerns)

הפרדת חששות היא עיקרון עיצובי שמטרתו לפצל את הקוד לחלקים נפרדים, כך שכל רכיב או שכבה מטפלים בתחום אחר של המערכת. הפרדה זו משפרת את הקריאות, התחזוקה והיכולת לשנות את המערכת בקלות.

ועל כן הקוד מחולק למספר שכבות לוגיות עצמאיות:

```
> analytics
> auth
> engine
> model
> ui
> utils
```

1. שכבת מנוע החישובים (Engine) ועיבוד הרקע

חלק עיקרי ומרכזי במערכת היא המחלקה *TrackingService* (עוד תוסבר בפירוט בהמשך) מחלקה זו פועלת מחוץ למחזור החיים הרגיל של המסכים הרגילים באפליקציה ומיושמת כ- **Foreground Service**

- **עקיפת מגבלות מערכת ההפעלה:** מערכות אנדרואיד מודרניות נוטות "להרוג" תהליכי רקע כדי לחסוך בסוללה. הגדרת הרכיב כ-

Foreground Service, בליווי התראה קבועה (Notification) בשורת המשימות, מבטיחה שמערכת ההפעלה תעניק לו עדיפות עליונה. כך מובטח שהאפליקציה תמשיך לדגום נתונים בצורה רציפה ואמינה גם במהלך ריצות שטח ארוכות, אימונים עצימים מתמשכים או הדמיות ארוכות, כאשר המכשיר נעול ונמצא בכיס.

- **גישה לחומרה:** השירות מנהל את האובייקט `SensorManager` ומקבל הרשאה מיוחדת-`HIGH_SAMPLING_RATE_SENSOR` כפי שצוין למעלה, כדי לדגום את מד התאוצה והגירוסקופ בתדר מקסימלי, החיוני לדיוק של האלגוריתם למדידת צעדים.

המחלקה לעומק:

```
/**
 * The primary execution engine of the application, running as an isolated Foreground Service.
 * This class encapsulates the Digital Signal Processing (DSP) pipeline and manages low-level
 * hardware hooks for the Accelerometer and Gyroscope sensors.
 * * Technical Design:
 * 1. Executes continuous background tracking independent of the UI thread to prevent frames dropping.
 * 2. Implements a Complementary Filter for real-time sensor fusion and posture analysis.
 * 3. Utilizes an optimized Discrete Fourier Transform (DFT) window for step frequency detection.
 * all explain in depth on the project book
 */
public class TrackingService extends Service implements SensorEventListener, TextToSpeech.OnInitListener {
```

אחריות המחלקה נחלקת לארבעה רכיבים טכניים מרכזיים:

א. מחזור חיים כשירות חזיתי (*Foreground Service Lifecycle*)

קריאת נתוני חיישנים ברזולוציה גבוהה דורשת משאבי סוללה ומעבד. במערכות אנדרואיד מודרניות, מערכת ההפעלה (OS) נוטה "להרוג" תהליכי רקע כדי לחסוך באנרגיה כדי למנוע את סגירת האימון בזמן שהטלפון בכיסו של המתאמן, המחלקה מיושמת כ **-Foreground Service**. עם הפעלת השירות, המערכת קוראת לפונקציה `startForeground()` ומציגה למשתמש התראה קבועה בשורת המשימות. פעולה זו מקנה לתהליך עדיפות (*Priority*) זהה לזו של אפליקציה הפתוחה על המסך וכן מערכת ההפעלה תעדיף שלא לסגור אותה ללא סיבה, ומבטיחה ריצה חלקה ובלתי מופרעת של אלגוריתם הניטור.

שם ההתראה:

```
2 usages
private static final String CHANNEL_ID = "InertiaTrackingChannel";
1 usage
private static final int NOTIFICATION_ID = 1;
```

יצירת ההתראה והפעלת התהליך כשירות חזיתי:

```
// 2. NORMAL STARTUP ROUTE: Establish VIP Foreground status
Notification notification = new NotificationCompat.Builder(context, CHANNEL_ID)
    .setContentTitle("Inertia Tracking")
    .setContentText("Workout in progress...")
    .setSmallIcon(android.R.drawable.ic_menu_compass)
    .setPriority(NotificationCompat.PRIORITY_LOW)
    .setOngoing(true)
    .build();

startForeground(NOTIFICATION_ID, notification);
```

ב. אינטגרציה עם חומרת המכשיר (Hardware Hooking)

המחלקה מנהלת את הגישה הישירה לחיישנים של המכשיר באמצעות האובייקט `SensorManager`.

```
sensorManager = (SensorManager) getSystemService(Context.SENSOR_SERVICE);
if (this.sensorManager != null) {
    this.accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);
    this.gyroscope = sensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE);
}
```

- **רישום למאזינים (*Listener Registration*):** המחלקה מיישמת את הממשק `SensorEventListener` ונרשמת (זוהי תבנית עיצוב מסוג צופה המיושמת כאן) לקבלת נתונים משני חיישנים מרכזיים: מד התאוצה הקווי (Linear Accelerometer) והגירוסקופ (Gyroscope).
- **קצב דגימה (*Sampling Rate*):** כדי שאלגוריתם התמרת פורייה (*DFT*) יוכל לזהות את תדר הצעדים בצורה חדה ולקיים את משפט נייקוויסט-שאנון (הקובע שצריך לדגום בתדר שגודלו לפחות פי שניים יותר מאשר התדר המקסימלי שנדגם, נובע ממשפטים בעיבוד אותות) הרישום לחיישנים מבוצע בקצב הגבוה ביותר האפשרי (`SENSOR_DELAY_FASTEST`) לשם כך, המערכת עושה שימוש בהרשאה הייעודית.

```
if (accelerometer != null && gyroscope != null) {
    sensorManager.registerListener(listener: this, accelerometer, SensorManager.SENSOR_DELAY_GAME);
    sensorManager.registerListener(listener: this, gyroscope, SensorManager.SENSOR_DELAY_GAME);
}
```

ג. ניהול זיכרון ואגירת נתונים (*Windowing & Buffering*)

מכיוון שקצב הדגימה עשוי להגיע למאות אירועים (*Events*) בשנייה, לא ניתן לבצע חישוב מתמטי כבד על כל דגימה בודדת.

המחלקה מיישמת תהליך של חלוקה לחלונות זמן

1. הנתונים הגולמיים הנקלטים בפונקציה `onSensorChanged()` אינם מעובדים מיד, אלא נאספים לתוך תור המאוחסן בזיכרון פנימי (Buffer). ישנו גם שימוש באריזה למחלקה פנימית.

```
/**
 * Callback interface triggered by the hardware sensor manager when a new kinetic event occurs.
 * Processes high-frequency raw data vectors, applies coordinate transformations,
 * and streams them through the internal mathematical filters.
 * pack the data on a custom class and sends it to processing.
 * @param event The SensorEvent object containing raw three-axis vectors (X, Y, Z).
 */
@Override
public void onSensorChanged(SensorEvent event) {
    if (event.sensor.getType() == Sensor.TYPE_GYROSCOPE) {
        this.gyroscopeDataForSynchronize[0] = event.values[0];
        this.gyroscopeDataForSynchronize[1] = event.values[1];
        this.gyroscopeDataForSynchronize[2] = event.values[2];
    } else if (event.sensor.getType() == Sensor.TYPE_ACCELEROMETER) {
        sensorQueue.offer(new SensorSample(
            event.values[0], event.values[1], event.values[2],
            this.gyroscopeDataForSynchronize[0], this.gyroscopeDataForSynchronize[1], this.gyroscopeDataForSynchronize[2],
            System.currentTimeMillis()
        ));
    }
}
```

2. כאשר המערך מתמלא לגודל המוגדר מראש (גודל החלון N , המותאם לאלגוריתם ה-DFT השירות נועל את אוגר הנתונים ומשגר את המערך המלא לפונקציות המתמטיות (המסנן המשלים ועיבוד האותות).

```
private void processSensorData() {
    if (this.sensorQueue.isEmpty()) return;

    int numberOfStepsDetected = 0, numberOfMagnitudeSamples = 256;
    SensorSample currentSample;
    ArrayList<Double> newMagnitudes = new ArrayList<>();

    while ((currentSample = this.sensorQueue.poll()) != null) {
        float x = currentSample.XLinearAcc;
        float y = currentSample.YLinearAcc;
        float z = currentSample.ZLinearAcc;
        double magnitude = Math.sqrt((x * x) + (y * y) + (z * z));

        this.dequeMagnitudes.addLast(magnitude);
        if (this.dequeMagnitudes.size() > numberOfMagnitudeSamples) {
            this.dequeMagnitudes.removeFirst();
        }

        // Execute Cadence Analysis when the window is full
        if (this.dequeMagnitudes.size() == numberOfMagnitudeSamples) {
            Double[] magnitudeArray = this.dequeMagnitudes.toArray(new Double[0]);
        }
    }
}
```

```
// Complementary Filter
double accelAngle = Math.atan2(y, Math.sqrt(z * z + x * x)) * (180.0 / Math.PI);
double alpha = 0.98;

if (lastSensorTime == 0) lastSensorTime = currentSample.timestamp;
double dt = (currentSample.timestamp - lastSensorTime) / 1000.0;
lastSensorTime = currentSample.timestamp;

this.currentAnglePitch = (this.currentAnglePitch + currentSample.gyroX * dt * (180.0 / Math.PI)) * alpha + (1 - alpha) * (accelAngle);
newMagnitudes.add(magnitude);
```

שיטת המסנן המשלים עובדת משיקולים מתמטיים, ובעיקרה היא אומרת כי לשלב מידע מכמה חיישנים ולתת אחוזים לכל אחד מהם עדיף מקבלת מידע על ידי חיישן אחד, בנוסף זה בא לפתור את הבעיה שחיישן ה GYRO מודד מהירות זוויתית במעלות לשנייה. כדי למצוא את הזווית עצמה, יש לבצע אינטגרציה (סכימה) של המהירות על פני זמן, הגירוסקופ מדויק מאוד בטווח הקצר ואינו מושפע מזעזועים קווים של צעדים. החיסרון שלו הוא **סחיפה** בגלל חוסר דיוק מיקרוסקופי בחומרה, האינטגרציה צוברת שגיאות זעירות בכל דגימה. כעבור מספר דקות, הזווית המחושבת תסטה משמעותית מהזווית האמיתית, גם אם המכשיר מונח ללא תזוזה.

ולכן ניתן להשתמש בממוצע האקספוננצילי שמופיע במקומות רבים בעולם המחשבים (חישוב זמן עיבוד על פי הזמני עיבוד הקודמים באלגוריתמי תזמון לדוגמה)

$$\theta_{pitch} = \alpha \cdot (\theta_{prev} + \omega_x \cdot \Delta t) + (1 - \alpha) \cdot \theta_{acc}$$

כאשר θ_{acc} זו הזווית שחושבה מוקטור כוח הכבידה של מד התאוצה.

ובכללי שיטה זו מאפשרת חישוב מחזורי יעיל בסיבוכיות של $O(N \cdot K)$ מבלי להציף את המעבד (CPU Throttling).

ד. תקשורת בין תהליכים (IPC - Inter-Process Communication)

לאחר שה TrackingService מסיים מחזור חישוב ובידיו הנתונים הפיזיקליים החדשים עליו להעביר אותם לשכבת התצוגה כדי שהמשתמש יראה אותם על המסך.

- התקשורת מתבצעת בצורה אסינכרונית ובטוחה באמצעות **BroadcastReceiver**.
- השירות אורז את התוצאות לתוך אובייקט מסוג *Intent* ומשדר אותו למערכת תחת ערוץ (Action) מוגדר היטב. מסך ה- *SensorTrackingActivity* מאזין לערוץ זה, שולף את הנתונים ומעדכן

את רכיבי ה-UI זה מבטיח שאפילו אם שכבת התצוגה קורסת או נסגרת, איסוף הנתונים באימון נמשך כסדרו ללא איבוד מידע

```
// Broadcast to UI
Intent updateIntent = new Intent(ACTION_WORKOUT_UPDATE);
updateIntent.setPackage(getPackageName());
updateIntent.putExtra(name: "steps", stepCount);
updateIntent.putExtra(name: "cadence", lastKnownCadence);
updateIntent.putExtra(name: "pitch", Math.round(currentAnglePitch));

double[] magsPrimitive = new double[newMagnitudes.size()];
for (int i = 0; i < newMagnitudes.size(); i++) magsPrimitive[i] = newMagnitudes.get(i);
updateIntent.putExtra(name: "magnitudes", magsPrimitive);

sendBroadcast(updateIntent);
}
```

המתודה `sendBroadcast()` מקבלת `Intent` ובצורה אסינכרונית שולחת את המידע וכך ממשיך הביצוע היא מטיפוס `void` ולכן גם לא מעדכנת האם המעוניינים במידע-אלה שיקבלו אותו, באמת קיבלו אותו, במקרה פה המידע נשלח ישירות למסך האימון.

2. שכבת הניתוח (Analytics Layer)

כדי לשמור על קוד נקי וניתן לבדיקה, הלוגיקה הספורטיבית והמתמטית מופרדת לחלוטין מתלויות של מערכת ההפעלה. (*Pure Java*) מחלקת ה-`WorkoutAnalyzer` משמשת כמערכת מומחה שמקבלת את הזרם הרציף של הנתונים הפיזיקליים וממירה אותם בזמן אמת לתובנות (כגון סיווג עצימות וחישוב ציון איכות לאימון) ומניחה בסיס להמשך הפיתוח באמצעות אלגוריתמי לימוד מכונה על נתונים מתויגים.

```
/**
 * The core analytics engine of the InertiaAI system.
 * This component acts as an Expert System within the Analytics Layer,
 * responsible for transforming raw, filtered physical variables into
 * actionable physiological insights and computational metrics.
 * * Architecture Principle: High Cohesion & Low Coupling - it contains zero
 * Android framework dependencies, making it pure, testable mathematical logic.
 */
3 pages
public class WorkoutAnalyzer {
```

3. שכבת התצוגה (UI Layer) ותקשורת בין תהליכים (IPC)

מסכי האפליקציה (ולדוגמא `SensorTrackingActivity` מתוכננים על פי תבנית "המסוף הטיפש" כל תפקידם הוא לרנדור את הממשק הגרפי ולהאזין לתוצאות.

- **תקשורת אסינכרונית:** מכיוון ששירות הרקע והממשק פועלים במקביל, העברת המידע ביניהם מתבצעת באמצעות מנגנון `BroadcastReceiver`. מנוע הרקע משדר באופן מחזורי אירועים (Intents) המכילים את המדדים המעובדים, (*Cadence, Pitch, Steps*), ושכבת ה-UI קולטת אותם ומעדכנת את רכיבי המסך בצורה בטוחה, מבלי לחסום את זרימת הנתונים.

4. שכבת מסד הנתונים והענן: ניהול מידע אסינכרוני ב-Google Cloud Firestore

בשלב תכנון המערכת הראשוני, נשקלה האפשרות להשתמש במסד נתונים מקומי מבוסס רכיבי חומרה במכשיר כגון `SQLite` עם זאת, מתוך הבנת קהל היעד של האפליקציה – מתאמנים המבצעים פעילות עצימה וזקוקים למעקב ארוך טווח, גיבוי נתונים חסין, ויכולת לגשת לתובנות האימון מכל מכשיר – הוחלט לשדרג את המערכת לארכיטקטורת ענן מבוססת על **Google Cloud Firestore**.

א. הצדקת הבחירה הטכנולוגית (NoSQL Document-Store)

`Google Cloud Firestore` הוא מסד נתונים מסוג `NoSQL` המאחסן מידע במבנה של מסמכים (*Documents*) המאוגדים בתוך אוספים (*Collections*) הבחירה במודל זה על פני מסד נתונים רלציוני (טבלאי) מציעה מספר יתרונות הנדסיים מובהקים לפרויקט:

1. **גמישות במבנה הנתונים (Schemaless):** אופי הפרויקט דינמי, ובעתיד מתוכננת הרחבה של רכיבי המידע (למשל, הוספת מערכי נקודות מפרקים ממסך מעבדת הווידאו שמתוכנן בעתיד). מסד `NoSQL` מאפשר להרחיב את מבנה המסמכים בצורה מאוד נוחה

2. **סנכרון אסינכרוני מובנה (Offline Persistence):** רצים ומתאמנים בשטח חווים לעיתים קרובות ניתוקי רשת או קליטה סלולרית חלשה. `Firestore` כולל מנגנון מטמון מקומי מובנה. כאשר המערכת מנסה לשמור אימון באזור ללא קליטה, ה-`SDK` שומר את המידע באופן זמני על זיכרון המכשיר, ומסנכרן אותו אוטומטית מול הענן ברגע שמתחדש החיבור לאינטרנט, באופן שקוף לחלוטין למשתמש וללא חסימת ליבות הכרחיות.

ב. חוזה המידע וארכיטקטורת ה-`POJO (Workout.java)`

הקשר בין הקוד בשפת *Java* לבין ייצוג הנתונים בענן מנוהל באמצעות **שכבת המודל** המיוצגת על ידי המחלקה *Workout* מחלקה זו מוגדרת כ-**POJO (Plain Old Java Object)**-מחלקה פשוטה.

- **תפקיד ארכיטקטוני:** המחלקה מגדירה את "חוזה המידע" הרשמי של המערכת. היא מכילה משתנים פרימיטיביים פרטיים המייצגים את מאפייני האימון: מזהה ייחודי, מזהה משתמש, חתימת זמן, כמות צעדים כוללת, ממוצע קצב צעדים, זווית יציבה ממוצעת וציון האימון המשוקלל.

```
/**
 * Domain Model Entity representing a compiled physical workout session.
 * This POJO (Plain Old Java Object) serves as the declarative schema contract for
 * cloud serialization and mapping within Google Cloud Firestore.
 * * * Serialization Design: Holds immutable primitive attributes representing both raw data aggregates
 * and analytical outputs generated by the analytics sub-systems.
 */
9 usages
public class Workout {

    2 usages
    private long timestamp;
    2 usages
    private int totalSteps;
    2 usages
    private int cadence;
    2 usages
    private int pitch;
    1 usage
}
```

- **מיפוי אובייקטים אוטומטי (Object Mapping):** השימוש בתוך המחלקה בשיטות גישה סטנדרטיות מאפשר ל *Firebase SDK*-לבצע פעולה של **Reflection** כאשר האפליקציה מעבירה את העצם *Workout* ישירות לענן, הספרייה מפרקת אותו אוטומטית למבנה ערך-מפתח תואם ללא צורך בכתיבת קוד המרה ידני ומסורבל.

ג. צנרת שליחת הנתונים (*The Upload Pipeline*)

כאשר המתאמן לוחץ על כפתור הסיום (*btnFinishWorkout*), האפליקציה מתחילה בתהליך השמירה:

```

private void stopTrackingAndSave() {

    if (sensorManager != null) sensorManager.unregisterListener(this);
    if (backgroundProcessor != null && !backgroundProcessor.isShutdown()) backgroundProcessor.shutdown();

    FirebaseAuth mAuth = FirebaseAuth.getInstance();
    if (mAuth.getCurrentUser() == null) {
        Log.e(tag: "TrackingService", msg: "CRITICAL ERROR: No user logged in.");
        broadcastSaveComplete(isSuccess: false);
        stopSelf();
        return;
    }

    String uid = mAuth.getCurrentUser().getUid();
    FirebaseFirestore db = FirebaseFirestore.getInstance();

    Map<String, Object> workoutData = new HashMap<>();
    workoutData.put("timestamp", System.currentTimeMillis());
    workoutData.put("total_steps", this.stepCount);
    workoutData.put("final_cadence_spm", this.lastKnownCadence);
    workoutData.put("final_pitch_angle", Math.round(this.currentAnglePitch));
    String intensity = WorkoutAnalyzer.getIntensityLevel(lastKnownCadence);
    int finalScore = WorkoutAnalyzer.calculateWorkoutScore(this.stepCount, this.lastKnownCadence);
    workoutData.put("workout_score", finalScore);
    workoutData.put("intensity_level", intensity);
    workoutData.put("final_pitch_angle", Math.round(this.currentAnglePitch));
    String intensity = WorkoutAnalyzer.getIntensityLevel(lastKnownCadence);
    int finalScore = WorkoutAnalyzer.calculateWorkoutScore(this.stepCount, this.lastKnownCadence);
    workoutData.put("workout_score", finalScore);
    workoutData.put("intensity_level", intensity);
    Log.d(tag: "TrackingService", msg: "Starting Firebase save process...");

    // A single, clean network call using addOnCompleteListener
    db.collection(collectionPath: "users").document(uid).collection(collectionPath: "workouts").add(workoutData).addOnCompleteListener(task -> {
        if (task.isSuccessful()) {
            speak(text: "Workout saved successfully");
            Log.d(tag: "TrackingService", msg: "Saved to cloud successfully!");
            broadcastSaveComplete(isSuccess: true);
            stopSelf();
        } else {
            Exception e = task.getException();
            String errorMsg = (e != null) ? e.getMessage() : "Unknown Error";
            Log.e(tag: "TrackingService", msg: "Firebase Task Failed! Reason: " + errorMsg);
            broadcastSaveComplete(isSuccess: false);
            stopSelf();
        }
    });
}

```

1. **אחזור מזהה משתמש:** המערכת פונה לרכיב *FirebaseAuth* ומחלצת את ה-*UID* הייחודי של המשתמש המחובר, כדי להבטיח הפרדת מידע מוחלטת בין משתמשי המערכת.
2. **יצירת הישות:** השירות אוסף את הפרמטרים המעובדים משירות הרקע ומהאנליטיקה, ויוצר מופע (*Instance*) חדש של *Workout*.

3. **הפעלה אסינכרונית (*Task-Based Execution*):** פנייה לענן מתבצעת ברקע כדי למנוע את קפיאת הממשק. המערכת משיגה הפניה לאוסף האימונים הכללי (*CollectionReference*) ומפעילה את המתודה *.add()*.

4. **ניהול מאזיני קצה (*Listeners*):** אל הפעולה משורשרים מאזיני הצלחה וכישלון:

- *addOnSuccessListener:* מופעל כאשר המידע נרשם בהצלחה (במכשיר או בענן), ודואג לסגור את מסך האימון ולהציג הודעת חיווי חיובית.

- *addOnFailureListener:* מופעל במקרה של שגיאת מערכת חריגה (כגון חריגת הרשאות אבטחה), ותפקידו לתעד את השגיאה (*Logging*) ולהתריע למשתמש על הבעיה מבלי להביא לקריסת האפליקציה.

נתוני הטבלה:

שם השדה	סוג נתונים	תיאור הפונקציה במערכת
<i>workoutId</i>	<i>String</i>	מזהה ייחודי אקראי המיוצר אוטומטית על ידי <i>Firestore</i> עבור הרשומה.
<i>userId</i>	<i>String</i>	מזהה המשתמש הייחודי (<i>UID</i>) שהופק ממערכת ה- <i>Authentication</i> .
<i>timestamp</i>	<i>Number / Timestamp</i>	הזמן המדויק שבו נשמר האימון (משמש למיון כרונולוגי במסך ההיסטוריה).
<i>avgCadence</i>	<i>Integer</i>	ממוצע ה- <i>Cadence</i> (צעדים לדקה).
<i>workoutScore</i>	<i>Integer</i>	הציון האנליטי הסופי שנקבע על ידי ה- <i>WorkoutAnalyzer</i> .
<i>avgPitch</i>	<i>Integer</i>	ממוצע זווית רכינת הגוף (במעלות) שנמדד באמצעות המסנן המשלים.

שכבת ההתחברות והרשמה (auth)

א. תהליך ההרשמה ווולידציה (SignUpActivity)

מסך זה מהווה את שער הכניסה למשתמשים חדשים. מעבר לעיצוב הממשק, המחלקה מיישמת לוגיקה של בקרת קלט (Input Validation) בטרם פנייה לשרת:

- **בדיקת תקינות:** המערכת מוודאת ששדות הדוא"ל והסיסמה אינם ריקים, ושהסיסמה עומדת בתקן מינימלי (אורך תווים).

```
// basic validation of the password (more then 6 characters are mandatory)
if (email.isEmpty() || password.length() < 6) {
    Toast.makeText(context: this, text: "Password must be at least 6 chars", Toast.LENGTH_SHORT).
    return;
}
```

- **יצירת חשבון אסינכרונית:** במידה והקלט תקין, המערכת מפעילה את הפונקציה `createUserWithEmailAndPassword`. פונקציה זו פונה אסינכרונית לשרתי `Google`.

```
/*
 *create from the FirebaseAuth instance's method user with email and password
 *the addOnCompleteListener method is making asynchronous behavior meaning its doing
 *this process on background process, this like a buzzer on a restaurant its when the
 *food is ready its buzz until then you wait but can do other things.
 *the createUserWithEmailAndPassword method return a Task this represent a asynchronous
 *and the addOnCompleteListener catch the task and then divide to cases, if the task did
 *it get the User Id that just have been created and call the private method that saves
 *base and add more information about the user.
 *else it is notify the user that the operation did not success (and show the reason).
 */
mAuth.createUserWithEmailAndPassword(email, password)
    .addOnCompleteListener( activity: this, task -> {
        if (task.isSuccessful()) {
            // get the UserId that just created
            String uid = mAuth.getCurrentUser().getUid();

            // call the method that saves the information and add more
            saveUserDetailsToDatabase(uid, email, fullName);
        }
        //failed to create the user with password from some reason, notify the user about
    })
    .addOnFailureListener( activity: this, task -> {
        //failed to create the user with password from some reason, notify the user about
    })
    .addOnCanceledListener( activity: this, task -> {
        //failed to create the user with password from some reason, notify the user about
    })
    .addOnCompleteListener( activity: this, task -> {
        //failed to create the user with password from some reason, notify the user about
    })
}
```

- **הצפנה (Cryptography):** היתרון המרכזי בשימוש ב `Firebase` הוא שהאפליקציה (צד הלקוח) אינה שומרת או מנהלת סיסמאות גלויות. ההצפנה, הגיבוב מתבצעים כולם בצד השרת של `Google` מה שמונע חשיפת נתונים במקרה של פריצה למכשיר הקצה.

ב. תהליך אימות משתמש קבוע (LoginActivity)

מסך ההתחברות מיועד למשתמשים חוזרים. הוא מקבל את פרטי ההזדהות ומפעיל את הפונקציה `signInWithEmailAndPassword`.

```
mAuth.signInWithEmailAndPassword(email, password)
    .addOnCompleteListener( activity: LoginActivity.this, task -> {
        if (task.isSuccessful()) {
            // success Login
            FirebaseUser user = mAuth.getCurrentUser();
            Toast.makeText( context: LoginActivity.this, text: "Login Successfully!", Toast.LENGTH_SHORT);

            //make an Intent to pass the user to the MainActivity
            Intent intent = new Intent(getApplicationContext(), MainActivity.class);
            //pass the user to the MainActivity
            startActivity(intent);

            //close the screen and not give the user the option to go back
            finish();
        } else {
            // failure
            // task.getException().getMessage()
            Toast.makeText( context: LoginActivity.this, text: "Invalid password or email", Toast.LENGTH_SHORT);
        }
    });
```

- **ניהול שגיאות (*Error Handling*):** מכיוון שהתקשורת תלויה ברשת, המחלקה מיישמת מאזינים (*Listeners*) המטפלים במקרי קצה. אם ההזדהות נכשלת (למשל, עקב סיסמה שגויה או משתמש שאינו קיים), המערכת קולטת את החריגה (*Exception*) ומציגה למשתמש משוב ברור ונקודתי מבלי לקרוס.

ג. ניהול סשן ואסימוני אבטחה (*Session Management & Tokens*)

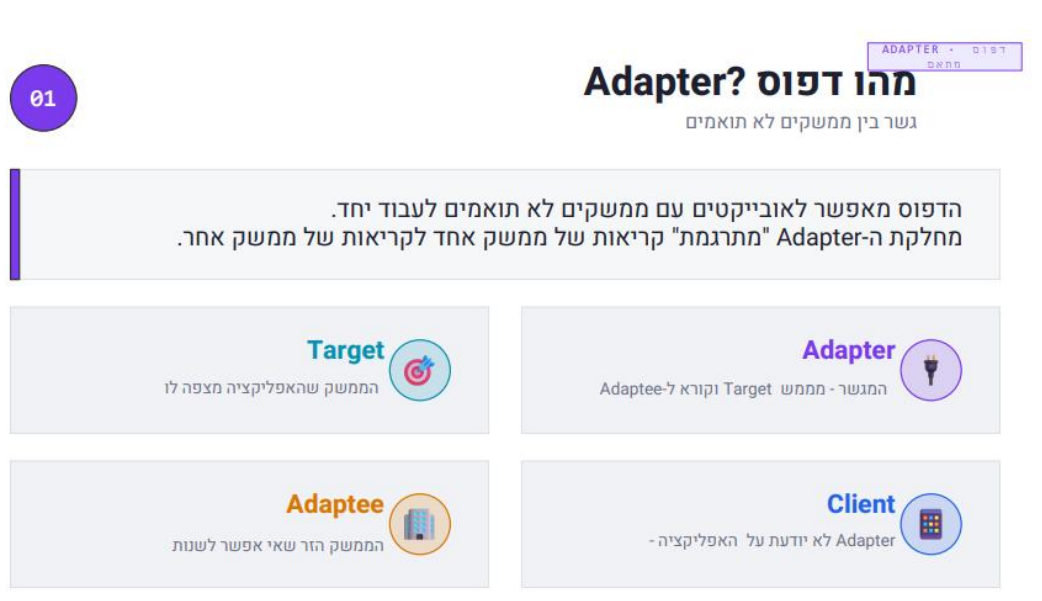
אחד האתגרים באפליקציות ענן הוא הצורך לדעת "מי המשתמש" בכל פעם שהוא פותח את האפליקציה או שומר אימון, מבלי לדרוש ממנו להקליד סיסמה מחדש.

- **Firestore** פותר זאת באמצעות מנגנון מבוסס אסימונים עם ההתחברות המוצלחת הראשונה, השרת מנפיק למכשיר אסימון גישה מאובטח הנשמר מקומית.
- כאשר נשמר אימון בענן (כפי שתואר בפרק מסד הנתונים), המערכת מושכת את ה-*UID (User ID)* הייחודי מתוך אובייקט הסשן הפעיל, ומצמידה אותו למסמך ה-*Workout* פעולה זו מבטיחה שאף משתמש לא יוכל למשוך או לצפות בהיסטוריית האימונים של משתמש אחר, תוך אכיפה קשיחה של כללי אבטחה.

שכבת התצוגה ועיבוד נתונים ויזואלי (UI Layer & Visual Data Representation)

החוליה האחרונה בארכיטקטורת ה-Client-Server של המערכת היא שכבת התצוגה. לאחר שהנתונים נאספו על ידי שירות הרקע, עובדו על ידי המערכת האנליטית, ונשמרו בענן, עולה הצורך להציג אותם למתאמן בצורה ברורה, מהירה, וללא פגיעה בביצועי המכשיר (למשל, תקיעות מסך בעת גלילה של היסטוריית אימונים ארוכה).

אתגר זה נפתר באמצעות שילוב של *HistoryActivity* ויישום תבנית העיצוב *Adapter*



א. משיכת הנתונים מהענן (*HistoryActivity*)

מסך זה פועל כ"בקר" של היסטוריית האימונים. תפקידו העיקרי הוא לנהל את התקשורת האסינכרונית מול בסיס הנתונים ולעדכן את ממשק המשתמש בהתאם.

- **שאלות אסינכרוניות:** עם פתיחת המסך, המערכת משגרת שאלתה ל *Firestore*-המבקשת לשלוח את כל מסמכי ה-*Workout*-השייכים ל-*UID* של המשתמש הנוכחי, ממוינים בסדר כרונולוגי יורד (מהאימון החדש לישן).

```

db.collection(collectionPath: "users").document(uid).collection(collectionPath: "workouts")
    .orderBy(field: "timestamp", Query.Direction.DESENDING)
    .get() Task<QuerySnapshot>
    .addOnSuccessListener(queryDocumentSnapshots -> {

        // Clear the old list to prevent duplicating data if we refresh
        workoutList.clear();

        // Loop through every document the cloud returned
        for (QueryDocumentSnapshot document : queryDocumentSnapshots) {

            Long timestamp = document.getLong(field: "timestamp");
            Long totalSteps = document.getLong(field: "total_steps");
            Long finalCadence = document.getLong(field: "final_cadence_spm");
            Long finalPitch = document.getLong(field: "final_pitch_angle");

            // Safety check: Prevent app crash if a field is missing in the database
            if (timestamp != null && totalSteps != null && finalCadence != null && finalPitch != null) {

                Workout workout = new Workout(
                    timestamp,
                    totalSteps.intValue(),
                    finalCadence.intValue(),
                    finalPitch.intValue()
                );
            }
        }
    })

```

• ניהול מצבי תצוגה: (UI States) מכיוון שמשיכת נתונים מהרשת אורכת

זמן, (Latency) המסך מנהל מצבי ביניים. בזמן ההמתנה מוצג חיווי טעינה מרכזי, (Progress Bar) ולאחר סיום השליפה, במידה והרשימה ריקה, תוצג הודעה מתאימה המזמינה את המשתמש לבצע את אימונו הראשון.

```

// --- UI UPDATE LOGIC ---
if (workoutList.isEmpty()) {
    // Empty State: Show the message, hide the list
    binding.rvWorkouts.setVisibility(android.view.View.GONE);
    binding.tvEmptyState.setVisibility(android.view.View.VISIBLE);
} else {
    // Data exists: Show the list, hide the message
    binding.rvWorkouts.setVisibility(android.view.View.VISIBLE);
    binding.tvEmptyState.setVisibility(android.view.View.GONE);

    // Initialize the adapter with the downloaded data and attach to RecyclerView
    WorkoutAdapter adapter = new WorkoutAdapter(workoutList);
    binding.rvWorkouts.setAdapter(adapter);
}
}

```

• ב. תבנית העיצוב מתאם (WorkoutAdapter)

כדי להציג רשימה דינמית שיכולה להכיל מאות אימונים, האפליקציה נמנעת משימוש ברכיבי רשימה ישנים (כגון ועושה שימוש ב- RecyclerView המודרני, אשר דורש יישום של מחלקת מתאם המחלקה WorkoutAdapter פועלת כגשר (Bridge) בין שכבת הנתונים

(מערך אובייקטי ה-*Workout* שהתקבלו מהענן) לבין שכבת התצוגה (קובץ ה-*XML* של כרטיסיית אימון בודדת).

```
/**
 * The Data-to-UI Bridge (Adapter Pattern) for the Workout History screen.
 * This class maps the raw 'Workout' data objects retrieved from the Cloud (Firestore)
 * into dynamic, memory-efficient visual view components (ViewHolders) within a RecyclerView.
 * * Performance Design: Implements the ViewHolder pattern to avoid costly 'findViewById'
 * operations during rapid scrolling, ensuring smooth 60fps rendering even with hundreds
 * of historical workout records.
 */
3 usages
public class WorkoutAdapter extends RecyclerView.Adapter<WorkoutAdapter.WorkoutViewHolder> {
```

- **ניהול זיכרון ומחזור תצוגות: (*View Recycling*)** היתרון ההנדסי המרכזי של המתאם הוא יעילות זיכרון קיצונית. במקום ליצור אובייקט גרפי חדש לכל אימון בהיסטוריה (פעולה יקרה שהייתה מציפה את זיכרון ה-*RAM* וגורמת לקריסות, המתאם מייצר מראש רק את כמות הכרטיסיות (*Views*) שנכנסות פיזית במסך (למשל 7-8 כרטיסיות). כאשר המשתמש גולל למטה, הכרטיסייה שעזבה את המסך למעלה לא נמחקת, אלא ממוחזרת (*Recycled*) ומועברת לתחתית הרשימה, כשהמתאם רק מחליף את הנתונים הטקסטואליים בתוכה.

ג. ארכיטקטורת ביצועים - תבנית *ViewHolder*

כדי להבטיח קצב רענון מסך אופטימלי של 60 פריימים בשנייה (60FPS) בזמן גלילה, המחלקה מיישמת מחלקה פנימית מסוג *WorkoutViewHolder*.

```
4 usages
static class WorkoutViewHolder extends RecyclerView.ViewHolder {

    5 usages
    private ItemWorkoutBinding binding;

    1 usage
    public WorkoutViewHolder(ItemWorkoutBinding binding) {
        super(binding.getRoot());
        this.binding = binding;
    }

    // Binds the Java object data to the XML TextViews
    1 usage
    public void bind(Workout workout) {
        binding.tvDate.setText(formatTimestamp(workout.getTimestamp()));
        binding.tvSteps.setText("Total Steps: " + workout.getTotalSteps());
        binding.tvCadence.setText("Average Cadence: " + workout.getCadence() + " SPM");
        binding.tvPitch.setText("Average Posture Angle: " + workout.getPitch() + "°");
    }

    /**
     Helper Method: Converts raw milliseconds to a human-readable date and time.
```

- תהליך הגישה לרכיבי ממשק בקוד (הפונקציה `findViewById()` הוא תהליך יקר מבחינה חישובית הדורש סריקה של עץ התצוגה (DOM) מחלקת ה `ViewHolder`-פועלת כמנגנון מטמון היא מחפשת את הרכיבים של תאריך, צעדים, וקצב פעם אחת בלבד בעת יצירת הכרטיסייה, ושומרת אליהם הפניות (*References*) ישירות בזיכרון.
- כך, בזמן שהמשתמש גולל את היסטוריית האימונים שלו במהירות, המערכת ניגשת ישירות לזיכרון לעדכון הנתונים, מה שמבטיח חוויית שימוש חלקה (*Smooth Scrolling*) ורספונסיבית לחלוטין.

הערה ארכיטקטונית על הטמעת מנגנון קישור תצוגה (*viewBinding*):

במסגרת פיתוח שכבת ממשק המשתמש של האפליקציה, הוחלט לעשות שימוש במנגנון מתקדם של קישור תצוגה. רכיב זה מחליף לחלוטין את הגישה המסורתית והישנה של חיפוש רכיבים ידני לפי מזהה, ומייצר באופן אוטומטי בזמן ההידור מחלקת קישור ייעודית עבור כל קובץ עיצוב פנימי. שילוב טכנולוגיה זו באפליקציה מונע לחלוטין באגים נפוצים בזמן ריצה, כגון הצבעות על ערכים ריקים עקב אי-קיום של רכיב במסך הנוכחי, ומבטיח בטיחות מלאה של סוגי הנתונים בשכבת הממשק תוך יעול משמעותי של מהירות רינדור הרכיבים.

```
binding on depth:
according to https://medium.com/@sandeepkella23/what-is-view-binding-in-android-and-how-it-works-i
and according to https://medium.com/@omersenturk.dev/3-ways-to-access-ui-components-in-android-pro
both binding and findViewById are ways to reference to a UI component
findViewById- do not support Null safety, means when a component does not exist and we
reference to it, this will cause crash on runtime.
also not support Type safety, if there is a mismatch between the Type of the component
will cause ClassCastException (another runtime).
every time when findViewById it scans all the full list of components.

each Layout file get a new class Named [LayoutFileName]Binding because of the gradle settings
hence, all the RuntimeException have no chance to occur, because it is an actual class and not
and if something does not exist or the Type is not correct, it will be on compile time.
also the search for the view will not occur during runtime but on compile time (on java way).
```

אתגרי פיתוח ופתרונם

במהלך תהליך התכנון והמימוש של המערכת, עלו מספר אתגרים הנדסיים מורכבים אשר דרשו פתרונות ארכיטקטוניים ואלגוריתמיים מתקדמים:

א. אתגר תאימות הגרסאות וניהול ספריות חיצוניות

בשלבי הפיתוח המוקדמים, שילוב ספריות הענן של פיירבייס יחד עם ספריות עזר חיצוניות הציג התנגשויות קשות בקובצי הגדרות הבנייה (גראדל). התנגשויות אלו מנעו את הידור התוכנה והובילו לשגיאות חסימה רבות.

- **הפתרון:** האתגר נפתר על ידי מעבר לעבודה עם ארכיטקטורת ניהול גרסאות מרכזית ואחידה המסופקת על ידי חברת גוגל. מנגנון זה מסנכרן ומאזן אוטומטית בין כל תתי-הספריות השונות של מסד הנתונים ומערכת האימות, ומבטיח עבודה מול גרסאות תואמות. בנוסף, בוצע ניקוי עמוק ובנייה מחדש של הפרויקט באופן מבוקר במערכת הבנייה.

ב. אתגר רעשי הרקע וסחיפת החיישנים בסביבה דינמית

ניסיון לחשב את זווית היציבה של המתאמן בזמן תנועה חשף מגבלות פיזיקליות קשות בחומרת המכשיר: מד התאוצה הציג תנודות חדות ורעשים קינטיים כתוצאה מהמכות של הצעדים בקרקע, ואילו חיישן המהירות הזוויתית (הגירוסקופ) סבל משגיאה מצטברת (סחיפה) שגרמה לנתונים לסטות מהמציאות ככל שחלף הזמן.

- **הפתרון:** הבעיה נפתרה על ידי פיתוח והטמעה של אלגוריתם היתוך חיישנים מתמטי המבוסס על מסנן משלים. המסנן שולב בקוד כך שהוא מפעיל מסנן מעביר נמוכים על מד התאוצה ומסנן מעביר גבוהים על הגירוסקופ. בצורה זו, המערכת מסתמכת על הדיוק לטווח הקצר של הגירוסקופ ומאזנת אותו לטווח הארוך אל מול וקטור כוח הכבידה של מד התאוצה, מה שמניב נתון יציבה נקי ויציב לחלוטין.

ג. אתגר הקפאת ממשק המשתמש בעת חישובים תכופים

הרצת אלגוריתם עיבוד אותות מורכב כמו התמרת פורייה בדידה על חלונות דגימה רחבים בזמן אמת, דרשה משאבי מעבד אינטנסיביים. ביצוע החישובים הללו על גבי חוט הממשק הראשי גרם לנפילת פריימים חדה, לתגובתיות איטית של הכפתורים ולסכנת קריסה של האפליקציה כולה.

- **הפתרון:** הלוגיקה המתמטית והפיזיקלית הופרדה לחלוטין משכבת התצוגה והועברה לתוך שירות חזיתי הפועל ברקע. השירות דוגם את החומרה ומבצע את התמרות הפורייה בנפרד, ורק כאשר מתקבלת תוצאה סופית היא משודרת בצורה אסינכרונית אל מסכי האפליקציה

באמצעות מנגנון משדר ומקלט פנימיים, מה ששומר על ממשק משתמש מהיר וחלק.

סיכום רפלקטיבי וכיווני פיתוח עתידיים

א. סיכום רפלקטיבי

פיתוח פרויקט זה היווה עבורי חוויה לימודית והנדסית יוצאת דופן. הפרויקט אילץ אותי לחבר וליישם בפועל תיאוריות מתמטיות מופשטות (כמו מנועי פירוק גלים וטורי פורייה מוכללים) ותיאוריות פיזיקליות (מכניקה קינטית והיתוך וקטורים) לתוך סביבת פיתוח תוכנה מעשית המוגבלת במשאבי קצה.

המערכת במצבה הנוכחי עומדת במלוא היעדים והדרישות שהוגדרו לה בדף האפיון: היא פותחת בהצלחה את ה"קופסה השחורה" של ניטור הספורט ומאפשרת למתאמנים לקבל משוב מדעי, איכותי ויציב על הישגיהם ועל מנח גופם, תוך שמירה על סנכרון רציף ומאובטח מול בסיס הנתונים בענן.

ב. כיווני פיתוח עתידיים

כפלטפורמה ארכיטקטונית המיועדת להתרחבות, המערכת מתוכננת לצמוח בשני כיוונים מרכזיים באבולוציה הבאה שלה:

1. **הפעלת מעבדת הווידאו:** מימוש מסך מעבדת הווידאו המתוכנן, אשר יעשה שימוש בספריות מקומיות של בינה מלאכותית וראייה ממוחשבת. המודול ינתח פריים אחר פריים מתוך סרטוני תרגילים שהמשתמש יעלה, ויחלץ מערך של שלושים ושלוש נקודות ציון המייצגות את מפרקי הגוף הגדולים.

2. **הטמעת מודל למידת מכונה:** נתוני התנועה הטהורים שנאספים כיום מחיישני המכשיר (קצב צעדים, תאוצות וזווית הטיה) ישולבו יחד עם זוויות המפרקים הדינמיות שיחולצו מהווידאו. מאגר מידע רחב ומתויג זה ישמש כבסיס לאימון מודל למידת מכונה מתקדם (כגון אלגוריתם יער אקראי בסביבת פייתון). המודל ידע להשוות את ביצועי המתאמן אל מול תבניות תנועה אופטימליות, לזהות חריגות בטכניקה (כמו קריסת ברכיים או גב עגול) ולספק משוב ביומכני אוטונומי וייחודי שיסייע במניעת פציעות עומס ובשיפור ההישגים הפיזיים.

קטעי קוד

AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission
android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission
android:name="android.permission.FOREGROUND_SERVICE_HEALTH" />
    <uses-permission
android:name="android.permission.HIGH_SAMPLING_RATE_SENSORS" />
    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication"
        tools:targetApi="31">
        <activity
            android:name=".ui.HistoryActivity"
            android:exported="false" />
        <activity
            android:name=".auth.LoginActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category
android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
            </activity>
        <activity
            android:name=".auth.SignupActivity"
            android:exported="false" />
        <activity
            android:name=".ui.MainActivity"
            android:exported="false"></activity>
        <activity
            android:name=".ui.SensorTrackingActivity"
            android:exported="false"></activity>

        <service
            android:name=".engine.TrackingService"
            android:exported="false"
            android:foregroundServiceType="health" />
    </application>
</manifest>

```

WorkoutAnalyzer.java

```

package com.example.myapplication.analytics;

/**
 * The core analytics engine of the InertiaAI system.

```

```

    * This component acts as an Expert System within the Analytics
    Layer,
    * responsible for transforming raw, filtered physical variables into
    * actionable physiological insights and computational metrics.
    * * Architecture Principle: High Cohesion & Low Coupling - it
    contains zero
    * Android framework dependencies, making it pure, testable
    mathematical logic.
    */
public class WorkoutAnalyzer {

    /**
     * Classifies the biometric physical strain and workout intensity
    based on step frequency.
     * Implements a deterministic rule-based classification model to
    map raw Cadence
     * (Steps Per Minute) into validated physiological activity
    zones.
     * Open to updates for more professional classification or to get
    more data and then classify.
     *
     * @param cadence The calculated cadence in Steps Per Minute
    (SPM).
     * @return A standardized string representing the user's current
    intensity zone.
     */
    public static String getIntensityLevel(int cadence) {
        if (cadence == 0) return "Resting";
        if (cadence < 100) return "Light Walk";
        if (cadence <= 130) return "Brisk Walk / Jogging";
        if (cadence <= 160) return "Running";
        return "Sprint / High Intensity";
    }

    /**
     * Computational scoring algorithm that rates the mathematical
    "quality" of a workout session.
     * Formulates a specialized fitness metric by normalizing total
    workload (steps),
     * factoring in effort multipliers (cadence), and applying
    explicit execution penalties
     * based on kinetic alignment and biomechanical posture errors
    (pitch drift).
     *
     * @param steps Total accumulated steps throughout the
    execution window.
     * @param cadence The calculated dominant step frequency (SPM).
     * @param pitch The average skeletal tilt/posture angle
    computed via sensor fusion.
     * @return An integer representing the normalized workout quality
    score (0 to 100+).
     */
    public static int calculateWorkoutScore(int steps, int cadence,
    double pitch) {
        int score = 0;

        int posturePenalty = Math.abs(pitch) > 15 ? 15 : 0;

        // give bonus to more cadence (greater pace)

```

```

        double cadenceMultiplier = cadence > 120 ? 1.5 : 1.0;

        // give a score according to the number of steps and the
        cadence and the posture
        score = (int) ((steps / 10.0) * cadenceMultiplier) -
        posturePenalty;

        return Math.max(0, score); // grade cannot be less than zero
    }
}

```

LoginActivity.java

```

package com.example.myapplication.auth;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.example.myapplication.R;
import com.example.myapplication.databinding.ActivityLoginBinding;
import com.example.myapplication.utils.ThemeManager;
import com.example.myapplication.ui.MainActivity;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;

//inherit from AppCompatActivity to get modern screen abilities
public class LoginActivity extends AppCompatActivity {
    //connected to the XML of this file, instead of using the
    findViewById method.
    /*

        binding on depth:
        according to https://medium.com/@sandeepkella23/what-is-view-binding-in-android-and-how-it-works-internally-74c9ce0e5422
        and according to https://medium.com/@omersenturk.dev/3-ways-to-access-ui-components-in-android-pros-and-cons-85364391b483
        both binding and findViewById are ways to reference to a UI
        component
        findViewById- do not support Null safety, means when a
        component does not exist and we
        reference to it, this will cause crash on runtime.
        also not support Type safety, if there is a mismatch between
        the Type of the component
        will cause ClassCastException (another runtime).
        every time when findViewById it scans all the full list of
        components.

        each Layout file get a new class Named
        [LayoutFileName]Binding because of the gradle settings
        hence, all the RuntimeException have no chance to occur,
        because it is an actual class and not
        and if something does not exist or the Type is not correct,
        it will be on compile time.
        also the search for the view will not occur during runtime
        but on compile time (on java way).
    */
}

```

```

    */
    ActivityLoginBinding binding;

    private FirebaseAuth mAuth;

    private ThemeManager themeManager;

    @Override
    //called when the screen is loaded
    protected void onCreate(Bundle savedInstanceState) {
        //need for the operating system of android to initialize the
screen
        super.onCreate(savedInstanceState);

        themeManager = new ThemeManager(this);
        themeManager.applyTheme(themeManager.isNightMode());

        //make the XML file from const and static on the UI to
dynamic with the binding
        this.binding =
ActivityLoginBinding.inflate(getLayoutInflater());

        //when doing UI operations the binding is the way to do that
        setContentView(binding.getRoot());

        // check for dark mode or light mode
        if (themeManager.isNightMode()) {
            //we are on dark mode so put the sun
binding.btnThemeToggle.setImageResource(R.drawable.ic_sun);
        } else {
            // we are on light mode so put the moon
binding.btnThemeToggle.setImageResource(R.drawable.ic_moon);
        }

        // explained on the SignupActivity
        this.mAuth = FirebaseAuth.getInstance();

        //when the login button is pressed do:
        this.binding.loginButton.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String email =
binding.loginEmail.getText().toString();
                String password =
binding.loginPassword.getText().toString();

                //if the email or the password field is empty make a
Toast.makeText (bubble notification) to the user
                if(email.isEmpty() || password.isEmpty())
                {
                    Toast.makeText(LoginActivity.this, "All fields
are mandatory", Toast.LENGTH_SHORT).show();
                }
            }
        });
    }

```

```

        else
        {

            mAuth.signInWithEmailAndPassword(email, password)

                .addOnCompleteListener(LoginActivity.this, task -> {
                    if (task.isSuccessful()) {
                        // success Login
                        FirebaseUser user =
mAuth.getCurrentUser();

Toast.makeText(LoginActivity.this, "Login Successfully!",
Toast.LENGTH_SHORT).show();

//make an Intent to pass the user
to the MainActivity

Intent intent = new
Intent(getApplicationContext(), MainActivity.class);
//pass the user to the
MainActivity

startActivity(intent);

//close the screen and not give
the user the option to go back

finish();
    } else {
        // failure
        //
task.getException().getMessage()

Toast.makeText(LoginActivity.this, "Invalid password or email",
Toast.LENGTH_SHORT).show();
    }
    });
        }
    });

    binding.btnThemeToggle.setOnClickListener(v -> {

        boolean newMode = !themeManager.isNightMode();

        themeManager.setNightMode(newMode);

        Intent intent = new Intent(LoginActivity.this,
LoginActivity.class);
        startActivity(intent);

        overridePendingTransition(R.anim.slide_in_right,
R.anim.slide_out_left);

        finish();
    });

    //when the Register an account button is pressed pass the
user to the SignUp activity

```

```

        binding.signupRedirectText.setOnClickListener(new
View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(getApplicationContext(),
SignupActivity.class);

        startActivity(intent);
        // called UI shit

overridePendingTransition(android.R.anim.slide_in_left,
android.R.anim.slide_out_right);
    }
});

binding.tvLoginTitle.setAlpha(0f);
binding.tvLoginTitle.setTranslationY(50);

binding.tvLoginSubtitle.setAlpha(0f);
binding.tvLoginSubtitle.setTranslationY(50);

binding.tilLoginEmail.setAlpha(0f);
binding.tilLoginEmail.setTranslationY(50);

binding.tilLoginPassword.setAlpha(0f);
binding.tilLoginPassword.setTranslationY(50);

binding.loginButton.setAlpha(0f);
binding.loginButton.setTranslationY(50);

binding.signupRedirectText.setAlpha(0f);
binding.signupRedirectText.setTranslationY(50);

int animationDuration = 500;

binding.tvLoginTitle.animate().alpha(1f).translationY(0).setDuration(
animationDuration).setStartDelay(100).start();

binding.tvLoginSubtitle.animate().alpha(1f).translationY(0).setDurati
on(animationDuration).setStartDelay(200).start();

binding.tilLoginEmail.animate().alpha(1f).translationY(0).setDuration
(animationDuration).setStartDelay(300).start();

binding.tilLoginPassword.animate().alpha(1f).translationY(0).setDurati
on(animationDuration).setStartDelay(400).start();

binding.loginButton.animate().alpha(1f).translationY(0).setDuration(a
nimationDuration).setStartDelay(500).start();

binding.signupRedirectText.animate().alpha(1f).translationY(0).setDurati
on(animationDuration).setStartDelay(600).start();
    }
}

```

SignupActivity.java

```
package com.example.myapplication.auth;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.example.myapplication.R;
import com.example.myapplication.databinding.ActivitySignupBinding;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.firestore.FirebaseFirestore;
import java.util.HashMap;
import java.util.Map;

public class SignupActivity extends AppCompatActivity {

    //FirebaseAuth is a class that follow the Singleton pattern
    //design, used to create/check for user on the database with methods.
    //can see on
    https://firebase.google.com/docs/reference/android/com/google/firebase/auth/FirebaseAuth#summary
    private FirebaseAuth mAuth;
    //also follow the Singleton pattern design, the access point to
    //the cloud database FireBase.
    private FirebaseFirestore db;
    //binding see LoginActivity to advantages over the normal way and
    //deep explanation.
    ActivitySignupBinding binding;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_signup);
        //the singleton behavior (need to get the only one instance)
        mAuth = FirebaseAuth.getInstance();
        db = FirebaseFirestore.getInstance();
        //normal binding inflation of the xml file to actual java
        class
        this.binding =
        ActivitySignupBinding.inflate(getLayoutInflater());
        //get all the information typed on UI component
        setContentView(binding.getRoot());
        binding.btnRegister.setOnClickListener(v -> {
            String email = binding.etEmail.getText().toString();
            String password =
            binding.etPassword.getText().toString();
            String fullName =
            binding.etFullName.getText().toString();

            // basic validation of the password (more then 6
            //characters are mandatory)
            if (email.isEmpty() || password.length() < 6) {
                Toast.makeText(this, "Password must be at least 6
                chars", Toast.LENGTH_SHORT).show();
                return;
            }
        });
    }
}
```

```

        }
        //call the private method defined above the register new
account.
        registerNewUser(email, password, fullName);
    });
    binding.tvLoginPrompt.setOnClickListener(new
View.OnClickListener() {
        @Override
        public void onClick(View v) {
            //Create an Intent to navigate from SignupActivity to
LoginActivity
            Intent intent = new Intent(SignupActivity.this,
LoginActivity.class);

            //Start the new activity
            startActivity(intent);
            // on simple words: UI shit

            overridePendingTransition(android.R.anim.slide_in_left,
android.R.anim.slide_out_right);

            // This prevents from the user from returning to the
Signup screen if they press the "Back" button on their phone
            finish();
        }
    });
    binding.tvTitle.setAlpha(0f);
    binding.tvTitle.setTranslationY(50);

    binding.tvSubtitle.setAlpha(0f);
    binding.tvSubtitle.setTranslationY(50);

    binding.tilFullName.setAlpha(0f);
    binding.tilFullName.setTranslationY(50);

    binding.tilEmail.setAlpha(0f);
    binding.tilEmail.setTranslationY(50);

    binding.tilPassword.setAlpha(0f);
    binding.tilPassword.setTranslationY(50);

    binding.btnRegister.setAlpha(0f);
    binding.btnRegister.setTranslationY(50);

    binding.tvLoginPrompt.setAlpha(0f);
    binding.tvLoginPrompt.setTranslationY(50);

    //some UI shit, not really important
    int animationDuration = 500;

    binding.tvTitle.animate().alpha(1f).translationY(0).setDuration(animationDuration).setStartDelay(100).start();

    binding.tvSubtitle.animate().alpha(1f).translationY(0).setDuration(animationDuration).setStartDelay(200).start();

    binding.tilFullName.animate().alpha(1f).translationY(0).setDuration(animationDuration).setStartDelay(300).start();

```

```

binding.tilEmail.animate().alpha(1f).translationY(0).setDuration(animationDuration).setStartDelay(400).start();

binding.tilPassword.animate().alpha(1f).translationY(0).setDuration(animationDuration).setStartDelay(500).start();

binding.btnRegister.animate().alpha(1f).translationY(0).setDuration(animationDuration).setStartDelay(600).start();

binding.tvLoginPrompt.animate().alpha(1f).translationY(0).setDuration(animationDuration).setStartDelay(700).start();
    }

    private void registerNewUser(String email, String password, String fullName) {
        /*
            *create from the FirebaseAuth instance's method user with email and password
            *the addOnCompleteListener method is making asynchronous behavior meaning its doing
            *this process on background process, this like a buzzer on a restaurant its when the
            *food is ready its buzz until then you wait but can do other things.
            *the createUserWithEmailAndPassword method return a Task this represent a asynchronous operation
            *and the addOnCompleteListener catch the task and then divide to cases, if the task did success then
            *it get the User Id that just have been created and call the private method that saves the details on the data
            *base and add more information about the user.
            *else it is notify the user that the operation did not success (and show the reason).
        */
        mAuth.createUserWithEmailAndPassword(email, password)
            .addOnCompleteListener(this, task -> {
                if (task.isSuccessful()) {
                    // get the UserId that just created
                    String uid = mAuth.getCurrentUser().getUid();

                    // call the method that saves the information and add more
                    saveUserDetailsToDatabase(uid, email, fullName);
                }
                //failed to create the user with password from some reason, notify the user about that
                else
                {
                    Toast.makeText(this, "Registration failed: " + task.getException().getMessage(), Toast.LENGTH_LONG).show();
                }
            });
    }

    private void saveUserDetailsToDatabase(String uid, String email, String fullName) {
        // make a collection (HashMap, similar to json) to store the user information

```

```

        Map<String, Object> userDetails = new HashMap<>();
        userDetails.put("email", email);
        userDetails.put("full_name", fullName);
        //save the data as Long (time is stored as Long on java)
        userDetails.put("joined_date", System.currentTimeMillis());
        // TODO: add more information about the user (Height, Weight
, etc ).

        /*
        FireBase is NoSQL database this means unlike the normal
SQL/SQLite it does not have
        tables and rows it build on collection and documents
        each document is key-value pair (like json) the pairs can
be from any type you need
        each document has a unique ID in his collection (in this
case set from the one that FireBase gave as default)
        a collection save a bunch of documents, you must put
documents inside collections
        */
        /*
        put on the users collection the document with user id,
uid
        the set method unlike the add method override the
existing data with the same uid
        https://firebase.google.com/docs/firestore/manage-
data/add-data#java
        return a task like the above method when the task is
finished
        */
        db.collection("users").document(uid)
            .set(userDetails)
            .addOnSuccessListener(aVoid -> {
                Toast.makeText(this, "Account Created!",
Toast.LENGTH_SHORT).show();
                //make an Intent to pass the user to the
LoginActivity
                Intent intent = new
Intent(getApplicationContext(), LoginActivity.class);
                //pass the user to the LoginActivity to sign in
with this user that just created
                startActivity(intent);
                //not give access to the user to go back to this
Activity
                finish();
            })
        /*
        CRITICAL DEBUGGING ADDITION:
        If the database write fails (e.g., due to
Security Rules or missing network),
        this listener will catch the exact reason and
output it to the user and Logcat.
        */
        .addOnFailureListener(e -> {
            android.util.Log.e("SignupActivity", "CRITICAL
DATABASE ERROR: " + e.getMessage());
            Toast.makeText(this, "Database Error: " +
e.getMessage(), Toast.LENGTH_LONG).show();
        });
    }
}

```

TrackingService.java

```
package com.example.myapplication.engine;

import android.app.Notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.Service;
import android.content.Context;
import android.content.Intent;
import android.hardware.Sensor;
import android.hardware.SensorEvent;
import android.hardware.SensorEventListener;
import android.hardware.SensorManager;
import android.os.Build;
import android.os.IBinder;
import android.speech.tts.TextToSpeech;
import android.util.Log;

import androidx.core.app.NotificationCompat;

import com.example.myapplication.analytics.WorkoutAnalyzer;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Locale;
import java.util.Map;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

/**
 * The primary execution engine of the application, running as an
 * isolated Foreground Service.
 * This class encapsulates the Digital Signal Processing (DSP)
 * pipeline and manages low-level
 * hardware hooks for the Accelerometer and Gyroscope sensors.
 * * Technical Design:
 * 1. Executes continuous background tracking independent of the UI
 * thread to prevent frames dropping.
 * 2. Implements a Complementary Filter for real-time sensor fusion
 * and posture analysis.
 * 3. Utilizes an optimized Discrete Fourier Transform (DFT) window
 * for step frequency detection.
 * all explain in depth on the project book
 */
public class TrackingService extends Service implements
    SensorEventListener, TextToSpeech.OnInitListener {

    // Identifiers required by the Android OS to maintain a
    Foreground Service state
    private static final String CHANNEL_ID =
    "InertiaTrackingChannel";
    private static final int NOTIFICATION_ID = 1;
```

```

    // Broadcast Action Strings - The "Radio Frequencies" used to
    communicate with the Activity
    public static final String ACTION_WORKOUT_UPDATE =
    "com.example.myapplication.WORKOUT_UPDATE";
    public static final String ACTION_STOP_AND_SAVE =
    "com.example.myapplication.STOP_AND_SAVE";
    public static final String ACTION_SAVE_COMPLETE =
    "com.example.myapplication.SAVE_COMPLETE";

    // --- Sensors and Hardware Access ---
    private SensorManager sensorManager;
    private Sensor accelerometer;
    private Sensor gyroscope;
    private TextToSpeech tts;

    // --- Thread-Safe Data Structures ---
    private ArrayBlockingQueue<SensorSample> sensorQueue;
    private float[] gyroscopeDataForSynchronize;
    private java.util.Deque<Double> dequeMagnitudes;

    // --- Algorithm & State Variables ---
    private int stepCount = 0;
    private int lastKnownCadence = 0;
    private long lastStepTime = 0;
    private long lastSensorTime = 0;
    private double currentAnglePitch = 0;

    private ScheduledExecutorService backgroundProcessor;

    // Biomechanical thresholds
    private static final double STEP_MOVMENT_TRESHHOLD = 3.5 + 9.8;
    private static final double STEP_TIME_TRESHHOLD = 300;
    private static final int MAX_ARRAY_SIZE = 1000;

    private class SensorSample {
        float XLinearAcc, YLinearAcc, ZLinearAcc, gyroX, gyroY,
gyroZ;
        long timeStamp;

        public SensorSample(float XLinearAcc, float YLinearAcc, float
ZLinearAcc, float gyroX, float gyroY, float gyroZ, long timeStamp) {
            this.XLinearAcc = XLinearAcc; this.YLinearAcc =
YLinearAcc; this.ZLinearAcc = ZLinearAcc;
            this.gyroX = gyroX; this.gyroY = gyroY; this.gyroZ =
gyroZ; this.timeStamp = timeStamp;
        }
    }

    @Override
    public void onCreate() {
        super.onCreate();

        createNotificationChannel();

        this.tts = new TextToSpeech(this, this);
        this.sensorQueue = new ArrayBlockingQueue<>(MAX_ARRAY_SIZE);
        this.gyroscopeDataForSynchronize = new float[3];
        this.dequeMagnitudes = new ArrayDeque<>(256);
    }

```

```

        sensorManager = (SensorManager)
getSystemService(Context.SENSOR_SERVICE);
        if (this.sensorManager != null) {
            this.accelerometer =
sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);
            this.gyroscope =
sensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE);
        }

@Override
public int onStartCommand(Intent intent, int flags, int startId)
{

    // 1. SAFE SHUTDOWN ROUTE
    if (intent != null &&
ACTION_STOP_AND_SAVE.equals(intent.getAction())) {
        stopTrackingAndSave();
        return START_NOT_STICKY;
    }

    // 2. NORMAL STARTUP ROUTE: Establish VIP Foreground status
    Notification notification = new
NotificationCompat.Builder(this, CHANNEL_ID)
        .setContentTitle("Inertia Tracking")
        .setContentText("Workout in progress...")
        .setSmallIcon(android.R.drawable.ic_menu_compass)
        .setPriority(NotificationCompat.PRIORITY_LOW)
        .setOngoing(true)
        .build();

    startForeground(NOTIFICATION_ID, notification);

    if (accelerometer != null && gyroscope != null) {
        sensorManager.registerListener(this, accelerometer,
SensorManager.SENSOR_DELAY_GAME);
        sensorManager.registerListener(this, gyroscope,
SensorManager.SENSOR_DELAY_GAME);
    }

    backgroundProcessor =
Executors.newSingleThreadScheduledExecutor();

backgroundProcessor.scheduleAtFixedRate(this::processSensorData, 0,
500, TimeUnit.MILLISECONDS);

    return START_STICKY;
}

/*
    The Consumer: Polls the queue safely, runs DFT (with Rest
Detection),
    applies the Complementary Filter, and broadcasts the
processed results.
*/
/*
    The Consumer: Polls the queue safely, runs DFT (with Rest
Detection),
    applies the Complementary Filter, and broadcasts the
processed results.

```

```

    */
    private void processSensorData() {
        if (this.sensorQueue.isEmpty()) return;

        int numberOfStepsDetected = 0, numberOfMagnitudeSamples =
256;
        SensorSample currentSample;
        ArrayList<Double> newMagnitudes = new ArrayList<>();

        while ((currentSample = this.sensorQueue.poll()) != null) {
            float x = currentSample.XLinearAcc;
            float y = currentSample.YLinearAcc;
            float z = currentSample.ZLinearAcc;
            double magnitude = Math.sqrt((x * x) + (y * y) + (z *
z));

            this.dequeMagnitudes.addLast(magnitude);
            if (this.dequeMagnitudes.size() >
numberOfMagnitudeSamples) {
                this.dequeMagnitudes.removeFirst();
            }

            // Execute Cadence Analysis when the window is full
            if (this.dequeMagnitudes.size() ==
numberOfMagnitudeSamples) {
                Double[] magnitudeArray =
this.dequeMagnitudes.toArray(new Double[0]);

                // --- Rest Detection (Prevents Phantom Cadence) ---
                double minMag = magnitudeArray[0];
                double maxMag = magnitudeArray[0];
                for (Double val : magnitudeArray) {
                    if (val < minMag) minMag = val;
                    if (val > maxMag) maxMag = val;
                }

                this.normalizeArray(magnitudeArray);

                // If there's barely any physical force difference,
the user is standing still
                if ((maxMag - minMag) < 2.0) {
                    this.lastKnownCadence = 0;
                } else {
                    this.lastKnownCadence =
this.getCadence(magnitudeArray, numberOfMagnitudeSamples, 50);
                }
            }

            // Complementary Filter
            double accelAngle = Math.atan2(y, Math.sqrt(z * z + x *
x)) * (180.0 / Math.PI);
            double alpha = 0.98;

            if (lastSensorTime == 0) lastSensorTime =
currentSample.timeStamp;
            double dt = (currentSample.timeStamp - lastSensorTime) /
1000.0;

            lastSensorTime = currentSample.timeStamp;

            this.currentAnglePitch = (this.currentAnglePitch +

```

```

currentSample.gyroX * dt * (180.0 / Math.PI)) * alpha + (1 - alpha) *
(accelAngle);

        newMagnitudes.add(magnitude);

        // Basic Step Detection
        if (magnitude > STEP_MOVMENT_TRESHHOLD &&
(currentSample.timeStamp - lastStepTime) > STEP_TIME_TRESHHOLD) {
            numberOfStepsDetected++;
            lastStepTime = currentSample.timeStamp;
        }
    }

    if (numberOfStepsDetected > 0) {
        stepCount += numberOfStepsDetected;
        if (stepCount % 10 == 0) {
            speak(stepCount + " steps completed");
        }
    }

    /*
    ANTI-GHOSTING SYSTEM:
    If more than 2 seconds (2000ms) have passed since the
last confirmed step,
    the user has likely stopped moving. We force the cadence
to 0 and clear
    the magnitude history to prevent the DFT algorithm from
analyzing old echoes.
    */
    if (System.currentTimeMillis() - lastStepTime > 2000) {
        this.lastKnownCadence = 0;
        this.dequeMagnitudes.clear();
    }

    // Broadcast to UI
    Intent updateIntent = new Intent(ACTION_WORKOUT_UPDATE);
    updateIntent.setPackage(getPackageName());
    updateIntent.putExtra("steps", stepCount);
    updateIntent.putExtra("cadence", lastKnownCadence);
    updateIntent.putExtra("pitch",
Math.round(currentAnglePitch));

    double[] magsPrimitive = new double[newMagnitudes.size()];
    for (int i = 0; i < newMagnitudes.size(); i++)
magsPrimitive[i] = newMagnitudes.get(i);
    updateIntent.putExtra("magnitudes", magsPrimitive);

    sendBroadcast(updateIntent);
}

/*
Clean, single-execution Firebase save method.
*/
private void stopTrackingAndSave() {

    if (sensorManager != null)
sensorManager.unregisterListener(this);
    if (backgroundProcessor != null &&
!backgroundProcessor.isShutdown()) backgroundProcessor.shutdown();
}

```

```

        FirebaseAuth mAuth = FirebaseAuth.getInstance();
        if (mAuth.getCurrentUser() == null) {
            Log.e("TrackingService", "CRITICAL ERROR: No user logged
in.");
            broadcastSaveComplete(false);
            stopSelf();
            return;
        }

        String uid = mAuth.getCurrentUser().getUid();
        FirebaseFirestore db = FirebaseFirestore.getInstance();

        Map<String, Object> workoutData = new HashMap<>();
        workoutData.put("timestamp", System.currentTimeMillis());
        workoutData.put("total_steps", this.stepCount);
        workoutData.put("final_cadence_spm", this.lastKnownCadence);
        workoutData.put("final_pitch_angle",
Math.round(this.currentAnglePitch));
        String intensity =
WorkoutAnalyzer.getIntensityLevel(lastKnownCadence);
        int finalScore =
WorkoutAnalyzer.calculateWorkoutScore(this.stepCount,
this.lastKnownCadence, this.currentAnglePitch);
        workoutData.put("workout_score", finalScore);
        workoutData.put("intensity_level", intensity);
        Log.d("TrackingService", "Starting Firebase save
process...");

        // A single, clean network call using addOnCompleteListener
        db.collection("users").document(uid).collection("workouts")
            .add(workoutData)
            .addOnCompleteListener(task -> {
                if (task.isSuccessful()) {
                    speak("Workout saved successfully");
                    Log.d("TrackingService", "Saved to cloud
successfully!");

                    broadcastSaveComplete(true);
                    stopSelf();
                } else {
                    Exception e = task.getException();
                    String errorMsg = (e != null) ?
e.getMessage() : "Unknown Error";
                    Log.e("TrackingService", "Firebase Task
Failed! Reason: " + errorMsg);
                    broadcastSaveComplete(false);
                    stopSelf();
                }
            });
    }

    private void broadcastSaveComplete(boolean isSuccess) {
        Intent completeIntent = new Intent(ACTION_SAVE_COMPLETE);
        completeIntent.putExtra("success", isSuccess);
        completeIntent.setPackage(getPackageName());
        sendBroadcast(completeIntent);
    }

    private void normalizeArray(Double[] arr) {
        double avg = 0.0;
        for (Double val : arr) avg += val;
    }

```

```

        avg /= arr.length;
        for (int i = 0; i < arr.length; i++) arr[i] -= avg;
    }

    /**
     * DSP Implementation: Extracts the dominant cadence frequency
     from time-domain acceleration arrays.
     * Transforms raw magnitude blocks into frequency-domain spectral
     coefficients using an
     * optimized  $O(N \cdot K)$  Discrete Fourier Transform (DFT) tuned
     explicitly for human locomotion frequencies (0.5Hz - 4.0Hz).
     *
     * @param arr The array of raw magnitude
     scalars.
     * @param numberOfMagnitudeSamples The exact window frame size
     (N).
     * @param numberOfSamplesPerSeconds The hardware sampling rate
     (Hz).
     * @return Calculated cadence value in Steps Per Minute (SPM).
     */
    private int getCadence(Double[] arr, int
numberOfMagnitudeSamples, int numberOfSamplesPerSeconds) {
        double maxCoefficient = 0.0, currentMagnitude;
        double maxK = 0.0;

        // We use a predefined factor to eliminate redundant division
        operations inside the heavy  $O(N \cdot K)$  loop
        double preCalculatedFactor = (2.0 * Math.PI) /
numberOfMagnitudeSamples;

        // Try to get the most accurate k value (amount of steps for
        about 5 seconds)
        //algorithm explained in depth on the project book
        for (double k = 5.0; k <= 25.0; k += 0.5) {
            double an = 0.0;
            double bn = 0.0;
            double kFactor = k * preCalculatedFactor;

            for (int n = 0; n < numberOfMagnitudeSamples; n++) {
                double angle = kFactor * n;
                an += arr[n] * Math.cos(angle);
                bn += arr[n] * Math.sin(angle);
            }
            currentMagnitude = Math.sqrt((an * an) + (bn * bn));
            if (currentMagnitude > maxCoefficient) {
                maxCoefficient = currentMagnitude;
                maxK = k;
            }
        }
        return (int) Math.round((60.0 * maxK *
numberOfSamplesPerSeconds) / numberOfMagnitudeSamples);
    }

    @Override
    public void onDestroy() {
        super.onDestroy();

        /*
        RESOURCE CLEANUP (Critical for avoiding Memory & Battery

```

```

Leaks):
    If the Android OS brutally kills this Service (e.g., user
    swipes app away),
    we MUST release the hardware sensors and shutdown the
    background thread,
    otherwise the phone's CPU will continue executing ghost
    tasks.
    */
    if (sensorManager != null) {
        sensorManager.unregisterListener(this);
    }

    if (backgroundProcessor != null &&
!backgroundProcessor.isShutdown()) {
        backgroundProcessor.shutdownNow();
    }

    if (tts != null) {
        tts.stop();
        tts.shutdown();
    }
}
/**
 * Callback interface triggered by the hardware sensor manager
 * when a new kinetic event occurs.
 * Processes high-frequency raw data vectors, applies coordinate
 * transformations,
 * and streams them through the internal mathematical filters.
 * pack the data on a custom class and sends it to processing.
 * @param event The SensorEvent object containing raw three-axis
 * vectors (X, Y, Z).
 */
@Override
public void onSensorChanged(SensorEvent event) {
    if (event.sensor.getType() == Sensor.TYPE_GYROSCOPE) {
        this.gyroscopeDataForSynchronize[0] = event.values[0];
        this.gyroscopeDataForSynchronize[1] = event.values[1];
        this.gyroscopeDataForSynchronize[2] = event.values[2];
    } else if (event.sensor.getType() ==
Sensor.TYPE_ACCELEROMETER) {
        sensorQueue.offer(new SensorSample(
            event.values[0], event.values[1],
event.values[2],
            this.gyroscopeDataForSynchronize[0],
this.gyroscopeDataForSynchronize[1],
this.gyroscopeDataForSynchronize[2],
            System.currentTimeMillis()
        ));
    }
}

@Override
public void onAccuracyChanged(Sensor sensor, int accuracy) { }

@Override
public void onInit(int status) {
    if (status == TextToSpeech.SUCCESS) {
        tts.setLanguage(Locale.US);
        speak("Tracking Started");
    }
}

```

```

    }
}

private void speak(String text) {
    if (tts != null) {
        tts.speak(text, TextToSpeech.QUEUE_FLUSH, null, null);
    }
}

private void createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        NotificationChannel serviceChannel = new
NotificationChannel(
            CHANNEL_ID,
            "Workout Tracking",
            NotificationManager.IMPORTANCE_LOW
        );

        getSystemService(NotificationManager.class).createNotificationChannel(
            serviceChannel);
    }
}

@Override
public IBinder onBind(Intent intent) {
    return null;
}
}

```

Workout.java

```

package com.example.myapplication.model;

/**
 * Domain Model Entity representing a compiled physical workout
 * session.
 * This POJO (Plain Old Java Object) serves as the declarative schema
 * contract for
 * cloud serialization and mapping within Google Cloud Firestore.
 * * * Serialization Design: Holds immutable primitive attributes
 * representing both raw data aggregates
 * and analytical outputs generated by the analytics sub-systems.
 */
public class Workout {

    private long timestamp;
    private int totalSteps;
    private int cadence;
    private int pitch;

    public Workout(long timestamp, int totalSteps, int cadence, int
pitch) {
        this.timestamp = timestamp;
        this.totalSteps = totalSteps;
        this.cadence = cadence;
        this.pitch = pitch;
    }
}

```

```

    public long getTimestamp() { return timestamp; }
    public int getTotalSteps() { return totalSteps; }
    public int getCadence() { return cadence; }
    public int getPitch() { return pitch; }
}

```

HistoryActivity.java

```

package com.example.myapplication.ui;

import android.os.Bundle;
import android.util.Log;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;

import com.example.myapplication.model.Workout;
import com.example.myapplication.databinding.ActivityHistoryBinding;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.Query;
import com.google.firebase.firestore.QueryDocumentSnapshot;

import java.util.ArrayList;
import java.util.List;

/**
 * HistoryActivity: The "Data Retrieval & Presentation" layer.
 * This Activity queries Cloud Firestore to retrieve all past
 * workouts
 * for the currently authenticated user and prepares them for the
 * RecyclerView.
 */
public class HistoryActivity extends AppCompatActivity {

    private ActivityHistoryBinding binding; // ViewBinding for null-
    safety
    private FirebaseFirestore db;
    private FirebaseAuth mAuth;

    // The internal memory list that will hold the downloaded cloud
    data
    private List<Workout> workoutList;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        binding =
ActivityHistoryBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());

        // Initialize Firebase singletons
        db = FirebaseFirestore.getInstance();
        mAuth = FirebaseAuth.getInstance();
        workoutList = new ArrayList<>();
    }

```

```

        // Setup the RecyclerView structure (Vertical scrolling list)
        binding.rvWorkouts.setLayoutManager(new
LinearLayoutManager(this));

        // Trigger the asynchronous network call
        fetchWorkoutHistory();
    }

    /**
     * Executes an asynchronous query to Firestore.
     * Fetches documents from the user's specific sub-collection,
     * ordered chronologically (newest first).
     */
    private void fetchWorkoutHistory() {
        if (mAuth.getCurrentUser() == null) {
            Toast.makeText(this, "Error: User not logged in",
Toast.LENGTH_SHORT).show();
            return;
        }

        String uid = mAuth.getCurrentUser().getUid();
        Log.d("HistoryActivity", "Fetching data for UID: " + uid);

        /**
         * Query Structure:
         * Go to users -> specific UID -> workouts collection.
         * Sort the results by the "timestamp" field in DESCENDING
order
         * so the most recent workout appears at the top of the
screen.
         */
        db.collection("users").document(uid).collection("workouts")
            .orderBy("timestamp", Query.Direction.DESENDING)
            .get()
            .addOnSuccessListener(queryDocumentSnapshots -> {

                // Clear the old list to prevent duplicating data
if we refresh
                workoutList.clear();

                // Loop through every document the cloud returned
                for (QueryDocumentSnapshot document :
queryDocumentSnapshots) {

                    Long timestamp =
document.getLong("timestamp");
                    Long totalSteps =
document.getLong("total_steps");
                    Long finalCadence =
document.getLong("final_cadence_spm");
                    Long finalPitch =
document.getLong("final_pitch_angle");

                    // Safety check: Prevent app crash if a field
is missing in the database
                    if (timestamp != null && totalSteps != null
&& finalCadence != null && finalPitch != null) {

                        Workout workout = new Workout(

```

```

        timestamp,
        totalSteps.intValue(),
        finalCadence.intValue(),
        finalPitch.intValue()
    );
    workoutList.add(workout);
    }
}

Log.d("HistoryActivity", "Successfully fetched "
+ workoutList.size() + " workouts.");

// --- UI UPDATE LOGIC ---
if (workoutList.isEmpty()) {
    // Empty State: Show the message, hide the
list
binding.rvWorkouts.setVisibility(android.view.View.GONE);
binding.tvEmptyState.setVisibility(android.view.View.VISIBLE);
} else {
    // Data exists: Show the list, hide the
message
binding.rvWorkouts.setVisibility(android.view.View.VISIBLE);
binding.tvEmptyState.setVisibility(android.view.View.GONE);

    // Initialize the adapter with the downloaded
data and attach to RecyclerView
    WorkoutAdapter adapter = new
WorkoutAdapter(workoutList);
    binding.rvWorkouts.setAdapter(adapter);
}
})
.addOnFailureListener(e -> {
    Log.e("HistoryActivity", "Failed to fetch
history: " + e.getMessage());
    Toast.makeText(this, "Failed to load workout
history.", Toast.LENGTH_SHORT).show();
});
}
}

```

MainActivity.java

```

package com.example.myapplication.ui;

import android.content.Intent;
import android.os.Bundle;

import androidx.appcompat.app.AppCompatActivity;

import com.example.myapplication.databinding.ActivityMainBinding;

public class MainActivity extends AppCompatActivity {

    private ActivityMainBinding binding;
}

```

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    // ViewBinding Initialization
    binding = ActivityMainBinding.inflate(getLayoutInflater());
    setContentView(binding.getRoot());

    // Navigate to the Live Sensor Tracking Engine
    binding.btnSensorLab.setOnClickListener(v -> {
        Intent intent = new Intent(MainActivity.this,
SensorTrackingActivity.class);
        startActivity(intent);
    });

    // Navigate to the Workout History (Cloud Data)
    binding.btnHistory.setOnClickListener(v -> {
        Intent intent = new Intent(MainActivity.this,
HistoryActivity.class);
        startActivity(intent);
    });
}
}

```

SensorTrackingActivity.java

```

package com.example.myapplication.ui;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;
import android.os.Build;
import android.os.Bundle;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.example.myapplication.engine.TrackingService;
import com.example.myapplication.databinding.ActivitySensorTrackingBinding;

/**
 * The View layer of the sensor tracking system.
 * Designed strictly under the Separation of Concerns (SoC)
principle, this class operates
 * as a passive, dumb terminal interface.
 * * Architectural Responsibilities:
 * 1. Dispatches lifecycle controls and commands to the background
execution service.
 * 2. Implements the Observer Pattern via an Inter-Process
Communication (IPC) BroadcastReceiver.
 * 3. Safely manipulates the UI thread to display real-time telemetry
using compiled ViewBinding mappings.
 */
public class SensorTrackingActivity extends AppCompatActivity {

    private ActivitySensorTrackingBinding binding;
}

```

```

        * Asynchronous Inter-Process Communication (IPC) Gateway.
        * Registers an explicit broadcast antenna on the OS level,
        capturing calculated physical metrics
        * emitted by the physics engine and pushing them safely into the
        main UI render queue.
        */
        private final BroadcastReceiver workoutReceiver = new
BroadcastReceiver() {
            @Override
            public void onReceive(Context context, Intent intent) {
                String action = intent.getAction();

                // EVENT 1: A live data update arrived from the physics
engine
                if (TrackingService.ACTION_WORKOUT_UPDATE.equals(action))
                {

                    int steps = intent.getIntExtra("steps", 0);
                    int cadence = intent.getIntExtra("cadence", 0);
                    int pitch = intent.getIntExtra("pitch", 0);

                    // Update the User Interface (TextViews)
                    binding.tvStepCount.setText("Steps: " + steps);

                    if (cadence > 0) {
                        binding.tvCadence.setText("Cadence: " + cadence +
" SPM");
                    }

                    binding.tvPosture.setText("Pitch Angle: " + pitch +
"°");

                    // חשבוניות שופרו UI-ה ביצועי - הוסר הגרף
                }
                // EVENT 2: The Service successfully uploaded the workout
to Firebase
                else if
                (TrackingService.ACTION_SAVE_COMPLETE.equals(action)) {
                    boolean isSuccess = intent.getBooleanExtra("success",
false);
                    if (isSuccess) {
                        Toast.makeText(context, "Workout saved securely
to the cloud!", Toast.LENGTH_LONG).show();
                    } else {
                        Toast.makeText(context, "ERROR: Not logged in or
network failed. Data lost.", Toast.LENGTH_LONG).show();
                    }
                    finish();
                }
            }
        };

        @Override
        protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);

            binding =
ActivitySensorTrackingBinding.inflate(getLayoutInflater());
            setContentView(binding.getRoot());

```

```

        binding.btnCalibrate.setOnClickListener(v -> {
            Toast.makeText(this, "Calibrated",
                Toast.LENGTH_SHORT).show();
        });

        binding.btnFinishWorkout.setOnClickListener(v -> {
            binding.btnFinishWorkout.setEnabled(false);
            binding.btnFinishWorkout.setText("SAVING...");

            Intent stopIntent = new Intent(this,
                TrackingService.class);
            stopIntent.setAction(TrackingService.ACTION_STOP_AND_SAVE);
            startService(stopIntent);
        });

        startTrackingService();
    }

    private void startTrackingService() {
        Intent serviceIntent = new Intent(this,
            TrackingService.class);
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            startForegroundService(serviceIntent);
        } else {
            startService(serviceIntent);
        }
    }

    @Override
    protected void onResume() {
        super.onResume();

        IntentFilter filter = new IntentFilter();
        filter.addAction(TrackingService.ACTION_WORKOUT_UPDATE);
        filter.addAction(TrackingService.ACTION_SAVE_COMPLETE);

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            registerReceiver(workoutReceiver, filter,
                Context.RECEIVER_NOT_EXPORTED);
        } else {
            registerReceiver(workoutReceiver, filter);
        }
    }

    @Override
    protected void onPause() {
        super.onPause();
        unregisterReceiver(workoutReceiver);
    }
}

```

WorkoutAdapter.java

```

package com.example.myapplication.ui;

import android.view.LayoutInflater;
import android.view.ViewGroup;

```

```

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.example.myapplication.model.Workout;
import com.example.myapplication.databinding.ItemWorkoutBinding;

import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;
import java.util.Locale;

/**
 * The Data-to-UI Bridge (Adapter Pattern) for the Workout History
 * screen.
 * This class maps the raw 'Workout' data objects retrieved from the
 * Cloud (Firestore)
 * into dynamic, memory-efficient visual view components
 * (ViewHolders) within a RecyclerView.
 * * Performance Design: Implements the ViewHolder pattern to avoid
 * costly 'findViewById'
 * operations during rapid scrolling, ensuring smooth 60fps rendering
 * even with hundreds
 * of historical workout records.
 */
public class WorkoutAdapter extends
RecyclerView.Adapter<WorkoutAdapter.WorkoutViewHolder> {

    private List<Workout> workoutList;

    public WorkoutAdapter(List<Workout> workoutList) {
        this.workoutList = workoutList;
    }

    /**
     * ViewHolder Inner Class: Acts as a high-performance cache.
     * It holds direct references to the UI elements within a single
workout card (XML),
     * preventing the system from re-parsing the layout files every
time a new row is drawn.
     */
    @NonNull
    @Override
    public WorkoutViewHolder onCreateViewHolder(@NonNull ViewGroup
parent, int viewType) {
        // Inflate the item layout using ViewBinding
        ItemWorkoutBinding binding = ItemWorkoutBinding.inflate(
            LayoutInflater.from(parent.getContext()),
            parent,
            false
        );
        return new WorkoutViewHolder(binding);
    }

    /**
     * Called to fill an existing card with specific workout data as
it scrolls into view.
     */
    @Override
    public void onBindViewHolder(@NonNull WorkoutViewHolder holder,
int position) {

```

```

        Workout currentWorkout = workoutList.get(position);
        holder.bind(currentWorkout);
    }

    @Override
    public int getItemCount() {
        return workoutList == null ? 0 : workoutList.size();
    }

    /**
     * ViewHolder: Caches the references to the UI components to
     * avoid calling findViewById repeatedly.
     */
    static class WorkoutViewHolder extends RecyclerView.ViewHolder {

        private ItemWorkoutBinding binding;

        public WorkoutViewHolder(ItemWorkoutBinding binding) {
            super(binding.getRoot());
            this.binding = binding;
        }

        // Binds the Java object data to the XML TextViews
        public void bind(Workout workout) {

            binding.tvDate.setText(formatTimestamp(workout.getTimestamp()));
            binding.tvSteps.setText("Total Steps: " +
            workout.getTotalSteps());
            binding.tvCadence.setText("Average Cadence: " +
            workout.getCadence() + " SPM");
            binding.tvPitch.setText("Average Posture Angle: " +
            workout.getPitch() + "°");
        }

        /**
         * Helper Method: Converts raw milliseconds to a human-
         * readable date and time.
         */
        private String formatTimestamp(long timestamp) {
            Date date = new Date(timestamp);
            SimpleDateFormat sdf = new SimpleDateFormat("dd MMM yyyy,
            HH:mm", Locale.getDefault());
            return sdf.format(date);
        }
    }
}

```

ThemeManager.java

```

package com.example.myapplication.utils;

import android.content.Context;
import android.content.SharedPreferences;
import androidx.appcompat.app.AppCompatActivity;
/**
 * Global Configuration Utility (Helper Pattern) responsible for
 * theme management.
 * This class centralizes the control and application of the
 * application's visual layout,

```

```

* specifically enforcing the high-contrast Dark Theme across all
decoupled Activities
* (e.g., MainActivity, SensorTrackingActivity, and HistoryActivity).
* * * Architectural Principle: Single Responsibility Principle (SRP)
- by isolating theme
* switching logic from individual UI controllers, it guarantees a
strict global visual
* baseline and prevents UI-state bugs during runtime configuration
changes.
*/
public class ThemeManager {
    private static final String PREF_NAME = "theme_prefs";
    private static final String KEY_IS_NIGHT_MODE = "isNightMode";
    private final SharedPreferences prefs;
    /**
     * Programmatically enforces the global visual styling
constraints onto a specified Context.
     * Dynamically injects color assets, window attributes, and
component styles (such as the
     * InertiaAI signature Cyan and Dark backgrounds-open for
changes) before the layout inflation phase.
     *
     * @param context The active Android Context (usually the calling
Activity) where the
     * visual theme properties must be applied.
     */

    public ThemeManager(Context context) {
        /*
         connect to the memory of the application
         this method get the name of the Preference in this case
theme_prefs
         to get the data for the theme (this is a file name)
         and the mode MODE_PRIVATE is the default for this use
case (just) get access with
         the user ID of this application (No other application
access this preference) to the file
         also this object is follow the singleton pattern design
and it is safe-thread

         */
        prefs = context.getSharedPreferences(PREF_NAME,
Context.MODE_PRIVATE);
    }

    // set the night mode to on or off and call other method
applyTheme
    //the apply method must be called to commit the changes.
    public void setNightMode(boolean isNight) {
        prefs.edit().putBoolean(KEY_IS_NIGHT_MODE, isNight).apply();
        applyTheme(isNight);
    }

    /**
     * Persists or evaluates the current user preference state
regarding the system's look and feel.
     * Acts as a gateway to check whether the application should
render in Dark Mode configuration,
     * maintaining consistency between device system settings and
cloud-native profile overrides.

```

```

    *
    * @return true if the system or user override mandates a dark
    visual configuration; false otherwise.
    */
    public boolean isNightMode() {
        return prefs.getBoolean(KEY_IS_NIGHT_MODE, true);
    }

    //this method just make all the application on night mode
    /*
    AppCompatActivity like the name , AppCompatActivity means support on the
    older versions of android
    AppCompatActivity take new features and translate them into older
    versions
    Delegate means, instead that all the activities will manage their
    UI/UX (in this case)
    they call an outside object to manage all this
    this is like a stage manager, when the app is running it call to
    colors.xml their night color are defined
    */
    public void applyTheme(boolean isNight) {
        if (isNight) {
AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_YES);
        } else {
AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_NO);
        }
    }
}

```

slide_in_right.xml

```

<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <translate
        android:duration="400"
        android:fromXDelta="100%"
        android:toXDelta="0%" />
</set>

```

slide_out_left.xml

```

<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <translate
        android:duration="400"
        android:fromXDelta="0%"
        android:toXDelta="-100%" />
</set>

```

activity_history.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#121212"> <TextView
    android:id="@+id/tvHistoryTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    android:text="Workout History"
    android:textColor="#00E5FF"
    android:textSize="28sp"
    android:textStyle="bold"
    android:letterSpacing="0.05"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />

    <View
        android:id="@+id/titleDivider"
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:layout_marginTop="16dp"
        android:background="#333333"
        app:layout_constraintTop_toBottomOf="@id/tvHistoryTitle" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvWorkouts"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:clipToPadding="false"
        android:paddingTop="8dp"
        android:paddingBottom="16dp"
        app:layout_constraintTop_toBottomOf="@id/titleDivider"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        tools:listitem="@layout/item_workout" />

    <TextView
        android:id="@+id/tvEmptyState"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="No workouts found.\nTime to go for a run!"
        android:textAlignment="center"
        android:textColor="#B0BEC5"
        android:textSize="18sp"
        android:visibility="gone"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

activity_login.xml

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="24dp"
    android:background="@color/screen_background"
    tools:context=".auth.LoginActivity">
    <ImageButton
        android:id="@+id/btnThemeToggle"
        android:layout_width="48dp"
        android:layout_height="48dp"
        android:background="?attr/selectableItemBackgroundBorderless"

        android:src="@drawable/ic_sun"

        app:tint="@color/brand_cyan"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_margin="16dp" />
    <TextView
        android:id="@+id/tvLoginTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="50dp"
        android:text="Welcome Back"
        android:textColor="@color/text_primary"
        android:textSize="32sp"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <TextView
        android:id="@+id/tvLoginSubtitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Sign in to continue your training"
        android:textColor="@color/text_secondary"
        android:textSize="16sp"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/tvLoginTitle" />
    <com.google.android.material.textfield.TextInputLayout
        android:id="@+id/tilLoginEmail"

style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="50dp"
        android:hint="Email"
        app:boxStrokeColor="@color/brand_cyan"
        app:hintTextColor="@color/brand_cyan">
```

```

        app:layout_constraintTop_toBottomOf="@id/tvLoginSubtitle">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/loginEmail"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textEmailAddress"
            android:textColor="@color/text_primary" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        android:id="@+id/tilLoginPassword"

style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:hint="Password"
        app:boxStrokeColor="@color/brand_cyan"
        app:hintTextColor="@color/brand_cyan"
        app:passwordToggleEnabled="true"
        app:passwordToggleTint="@color/text_secondary"
        app:layout_constraintTop_toBottomOf="@id/tilLoginEmail">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/loginPassword"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPassword"
            android:textColor="@color/text_primary" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.button.MaterialButton
        android:id="@+id/loginButton"
        android:layout_width="match_parent"
        android:layout_height="60dp"
        android:layout_marginTop="40dp"
        android:text="LOGIN"
        android:textSize="18sp"
        android:textStyle="bold"
        app:backgroundTint="@color/brand_cyan"
        app:cornerRadius="8dp"
        app:layout_constraintTop_toBottomOf="@id/tilLoginPassword" />

    <TextView
        android:id="@+id/signupRedirectText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:text="Don't have an account? Sign Up"
        android:textColor="@color/brand_cyan"
        android:textSize="16sp"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/loginButton" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#121212"
    tools:context=".ui.MainActivity">

    <TextView
        android:id="@+id/appTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="100dp"
        android:text="INERTIA AI"
        android:textColor="#00E5FF"
        android:textSize="36sp"
        android:textStyle="bold"
        android:letterSpacing="0.1"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/appSubtitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Smart Motion & Tracking"
        android:textColor="#B0BEC5"
        android:textSize="16sp"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/appTitle" />

    <Button
        android:id="@+id/btnSensorLab"
        android:layout_width="0dp"
        android:layout_height="80dp"
        android:layout_marginStart="32dp"
        android:layout_marginTop="100dp"
        android:layout_marginEnd="32dp"
        android:backgroundTint="#1E88E5"
        android:text="Live Sensor Tracking"
        android:textSize="18sp"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/appSubtitle" />

    <Button
        android:id="@+id/btnHistory"
        android:layout_width="0dp"
        android:layout_height="80dp"
        android:layout_marginStart="32dp"
        android:layout_marginTop="24dp"
        android:layout_marginEnd="32dp"
```

```

        android:backgroundTint="#FF8F00"
        android:text="Workout History"
        android:textSize="18sp"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/btnSensorLab" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

activity_sensor_tracking.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    android:background="#121212">
    <com.google.android.material.card.MaterialCardView
        android:id="@+id/cardStats"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        app:cardBackgroundColor="#1E1E1E"
        app:cardCornerRadius="12dp"
        app:cardElevation="4dp"
        app:strokeWidth="1dp"
        app:strokeColor="#00E5FF"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <TextView
                android:id="@+id/tvStepCount"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Steps: 0"
                android:textColor="#FFFFFF"
                android:textSize="24sp"
                android:textStyle="bold"
                android:layout_marginBottom="8dp" />

            <TextView
                android:id="@+id/tvCadence"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Cadence: -- SPM"
                android:textColor="#00E5FF"
                android:textSize="20sp"
                android:textStyle="bold"

```

```

        android:layout_marginBottom="8dp" />

        <TextView
            android:id="@+id/tvPosture"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Pitch Angle: 0°"
            android:textColor="#B0BEC5"
            android:textSize="18sp" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnCalibrate"
        android:layout_width="0dp"
        android:layout_height="55dp"
        android:layout_marginEnd="8dp"
        android:layout_marginBottom="16dp"
        android:text="CALIBRATE"
        app:backgroundTint="#333333"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toStartOf="@id/btnFinishWorkout" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnFinishWorkout"
        android:layout_width="0dp"
        android:layout_height="55dp"
        android:layout_marginStart="8dp"
        android:layout_marginBottom="16dp"
        android:text="FINISH"
        android:textStyle="bold"
        app:backgroundTint="#FF5252"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/btnCalibrate" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

activity_signup.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="24dp"
    android:background="@color/screen_background"
    tools:context=".auth.SignupActivity">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="32dp"
        android:text="Create Account"
        android:textColor="@color/text_primary"
    >

```

```

        android:textSize="32sp"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

<TextView
    android:id="@+id/tvSubtitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="Join Inertia Tracking today"
    android:textColor="@color/text_secondary"
    android:textSize="16sp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tvTitle" />

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/tilFullName"

style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="40dp"
    android:hint="Full Name"
    app:boxStrokeColor="@color/brand_cyan"
    app:hintTextColor="@color/brand_cyan"
    app:layout_constraintTop_toBottomOf="@id/tvSubtitle">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etFullName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPersonName"
        android:textColor="@color/text_primary" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/tilEmail"

style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:hint="Email"
    app:boxStrokeColor="@color/brand_cyan"
    app:hintTextColor="@color/brand_cyan"
    app:layout_constraintTop_toBottomOf="@id/tilFullName">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress"
        android:textColor="@color/text_primary" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/tilPassword"

```

```

style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:hint="Password"
    app:boxStrokeColor="@color/brand_cyan"
    app:hintTextColor="@color/brand_cyan"
    app:passwordToggleEnabled="true"
    app:passwordToggleTint="@color/text_secondary"
    app:layout_constraintTop_toBottomOf="@id/tilEmail">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPassword"
        android:textColor="@color/text_primary" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.button.MaterialButton
    android:id="@+id/btnRegister"
    android:layout_width="match_parent"
    android:layout_height="60dp"
    android:layout_marginTop="32dp"
    android:text="SIGN UP"
    android:textSize="18sp"
    android:textStyle="bold"
    app:backgroundTint="@color/brand_cyan"
    app:cornerRadius="8dp"
    app:layout_constraintTop_toBottomOf="@id/tilPassword" />

<TextView
    android:id="@+id/tvLoginPrompt"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    android:text="Already have an account? Login"
    android:textColor="@color/brand_cyan"
    android:textSize="16sp"
    android:textStyle="bold"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/btnRegister" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

item_workout.xml

```

<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginHorizontal="16dp"
    android:layout_marginVertical="8dp"
    app:cardBackgroundColor="#1E1E1E"

```

```

app:cardCornerRadius="12dp"
app:cardElevation="4dp"
app:strokeWidth="1dp"
app:strokeColor="#333333">

<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvDate"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:textColor="#FFFFFF"
        android:textSize="18sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        tools:text="June 02, 2026 - 17:30" />

    <View
        android:id="@+id/divider"
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:layout_marginTop="8dp"
        android:background="#333333"
        app:layout_constraintTop_toBottomOf="@id/tvDate" />

    <TextView
        android:id="@+id/tvSteps"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:textColor="#B0BEC5"
        android:textSize="16sp"
        app:layout_constraintTop_toBottomOf="@id/divider"
        app:layout_constraintStart_toStartOf="parent"
        tools:text="Total Steps: 1,450" />

    <TextView
        android:id="@+id/tvCadence"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:textColor="#00E5FF"
        android:textSize="16sp"
        android:textStyle="bold"
        app:layout_constraintTop_toBottomOf="@id/tvSteps"
        app:layout_constraintStart_toStartOf="parent"
        tools:text="Average Cadence: 164 SPM" />

    <TextView
        android:id="@+id/tvPitch"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:textColor="#B0BEC5"
        android:textSize="16sp"

```

```

        app:layout_constraintTop_toBottomOf="@id/tvCadence"
        app:layout_constraintStart_toStartOf="parent"
        tools:text="Average Posture Angle: 5°" />

    </androidx.constraintlayout.widget.ConstraintLayout>

</com.google.android.material.card.MaterialCardView>

```

build.gradle.kts(:app)

```

plugins {
    alias(libs.plugins.androidApplication)
    id("com.google.gms.google-services")
}

android {
    namespace = "com.example.myapplication"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.example.InertiaAI"
        minSdk = 24
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner =
"androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-
optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    buildFeatures {
        viewBinding = true
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_1_8
        targetCompatibility = JavaVersion.VERSION_1_8
    }
}

dependencies {
    // 1. Android Core & UI
    implementation(libs.appcompat)
    implementation(libs.material)
    implementation(libs.activity)
    implementation(libs.constraintlayout)

    // 2. Firebase BOM (Bill of Materials) (not the newest
version but, stable one)

```

```
        implementation(platform("com.google.firebase:firebase-  
bom:32.7.0"))  
  
        // 3. Firebase Services  
        implementation("com.google.firebase:firebase-analytics")  
        implementation("com.google.firebase:firebase-firestore")  
        implementation("com.google.firebase:firebase-auth")  
  
        // 4. AndroidPlot  
        implementation("com.androidplot:androidplot-core:1.5.10")  
  
        // 6. Testing  
        testImplementation(libs.junit)  
        androidTestImplementation(libs.ext.junit)  
        androidTestImplementation(libs.espresso.core)  
    }  
}
```